

Benchmarking

A benchmark tells you only how fast the system ran the benchmark. Here is what that number actually is, the sixteen ways it lies, how to choose the kind you need — and the one habit that catches most of it.

1

What the number actually is

Why you benchmark, what makes one good, and the four things you must understand.

2

The sixteen ways it lies

From measuring the wrong thing by accident to engineering the product to score.

3

Choosing the kind

Micro-benchmark, simulation, replay, industry standard — and what each costs.

4

Analyse it while it runs

Passive versus active, the checklist, and the questions to ask a vendor.

Print double-sided, short-edge bind, A4 · 100% scale (no “fit to page”).

Answer key starts at Section 9 — keep a sheet of paper over it.

How to use this booklet

The loop, per section

- 1. Read the plain-version box first. If it doesn't land, the rest won't either.
- 2. Read the teaching. Say each boundary out loud: “X is *this*, its look-alike Y is *that*.”
- 3. Cover the page. Write the answer to **Q1** in the blank lines, in your own words, in full sentences. **Then** check the key.
- 4. Score yourself on completeness. Then do **Q2** and note whether it beat Q1.
- 5. Only then move on.

THE RULE THAT DOES MOST OF THE WORK

If you find yourself thinking “yes, I know this” — that is *recognition*, and it is not learning. The only evidence that counts is a written answer produced with the page covered.

WHAT THIS SUBJECT ASKS OF YOU THAT OTHERS DON'T

Almost every page here is about **disbelieving a number you already have**. That is an unusual skill, because the number arrives looking finished: it has units, it has decimal places, and somebody has already put it on a slide. Nothing about it announces that the test measured a cache rather than a disk, or that every request errored, or that the tool itself was the bottleneck. So as you read, practise the specific move this competency is made of — take a plausible result and ask, out loud, *what would have to be true for this to be wrong?* The failures in Part 2 are all answers to that question that somebody once failed to ask.

The spacing schedule

Review at **day 1, day 3, day 7, day 16, day 35**. On a pass, jump to the next interval. On a fail, reset that item to 1 day. Use the grid at the back; one row per item.

What's in here

1 — Learning goal and roadmap	orientation
2 — What the number actually is	teaching · questions · matrix
3 — The sixteen ways it lies	teaching · questions · matrix
4 — Choosing the kind	teaching · questions · matrix
5 — Analyse it while it runs	teaching · questions · matrix
6 — Concept sketch + master matrix	consolidation
7 — Symptom → diagnosis → move · reverse mapping	working reference
8 — Numbers worth remembering · Glossary	reference
9 — Answer key	keep covered
10 — Spaced-review tracker · final quiz	workbook

Learning goal & roadmap

LEARNING GOAL — WHAT YOU SHOULD BE ABLE TO DO WHEN YOU CLOSE THIS

Handed a benchmark result — yours, a colleague's, or a vendor's — you can:

1. **Say what was actually measured**, as distinct from what the tool's documentation claims it measures.
2. **Name the limiting factor**, or say that nobody established one — which is the same as saying the system was never driven to its limit.
3. **Choose the right kind of benchmark** for the question being asked, and state what that kind cannot tell you.
4. **Ask the questions that expose the rest**: the configuration, the duration, the error rate, the variance, the distribution, and the price.

The core model in one paragraph

A benchmark result is not a fact about your system. It is a fact about **how fast the system ran that benchmark**, and everything that makes it useful — or worthless — is in the gap between those two statements. The gap has three sources. First, the test may not measure what its name says: tools that claim to test disks routinely test the file system cache instead, and the difference is orders of magnitude. Second, the run may not have reached the thing you meant to measure: it may have been limited by the benchmark software, by an unrelated congested component, by a configuration you did not check, or by errors that returned instantly and were counted as successes. Third, the result may be true and still misleading, because the price, the configuration, or the fact that the product was engineered specifically to score well on this test were left out. The defence against all three is the same and is simple to state: **analyse the system while the benchmark is still running**. Everything else in these pages is either a catalogue of what goes wrong when you don't, or a technique for doing it well.

TENTH-GRADER VERSION — THE WHOLE THING AT ONCE

- A benchmark measures how fast your computer does *the thing the benchmark does*. That may not be the thing you care about.
- The classic mistake: you think you measured A, you actually measured B, and you tell everyone you measured C.
- Numbers look finished. They have units and decimal points. Nothing about them warns you that the test never touched the disk.
- So the fix isn't a better tool — it's watching the machine while the test runs, and asking what stopped it going faster.
- And when someone hands you a number, the useful question isn't “is that good?” It's “what was it, exactly, that you measured?”

The four parts, and why they're in this order

PART	WHAT IT COVERS	WHY HERE
1 What the number actually is	The reasons benchmarking is done at all; the qualities of a good benchmark and the price/performance ratio; the four things you must understand about any result; and what a well-analysed result looks like when someone does it properly.	You cannot detect a bad result without a picture of a good one. This part supplies the standard the rest is measured against.
2 The sixteen ways it lies	Casual benchmarking, blind faith, numbers without analysis, complex tools, testing the wrong thing, ignoring the environment, errors, variance and perturbations, changing multiple factors, the benchmark paradox, benchmarking the competition, friendly fire, misleading results, benchmark specials, and cheating.	A catalogue, not a lecture. Most of these are ordinary, unglamorous and accidental — which is exactly why they survive review.
3 Choosing the kind	Micro-benchmarks and how to design a matrix of them; simulations, stateless and stateful; trace replay and why it disappoints; industry-standard benchmarks and what their disclosures give you.	Several of Part 2's failures are really the wrong <i>kind</i> of benchmark chosen for the question. This is where that choice gets made deliberately.
4 Analyse it while it runs	Passive benchmarking as the anti-method; active benchmarking with a worked case; profiling; applying the resource sweep and workload characterization; custom benchmarks; ramping load; the sanity check; statistical analysis; and the question lists for results you cannot analyse yourself.	The cure. One habit — observing the system during the run — prevents or exposes the majority of what Part 2 catalogues.

“There are lies, damn lies and then there are performance measures.”

1

PART 1 OF 4

What the number actually is

What a benchmark is for, what makes one good, and the four things you must understand before quoting a result.

TENTH-GRADER VERSION

- People run benchmarks for seven different reasons, and the reason changes what a good answer looks like.
- A good benchmark can be repeated, watched while it runs, moved to another machine, explained to your boss, and finished quickly — and it has to resemble something real.
- Before you quote a number you need four things: what was tested, what stopped it going faster, what else might have interfered, and what you're actually allowed to conclude.
- The most common way to get all four wrong is to run the test, walk away, and ask an expert to explain the numbers afterwards.
- By then the machine is idle and nobody can check anything.

THE EVERYDAY VERSION

Testing a car's fuel economy. The number on the sticker is real — somebody genuinely measured it — but it was measured on a rolling road, at a fixed temperature, with the air conditioning off and a professional driver holding a smooth throttle. None of that is a lie. It is also not your commute. The useful questions are the same four: *what exactly was tested* (a standardised cycle, not traffic), *what limited it* (the cycle's own speed profile, not the engine), *what might have perturbed it* (temperature, tyre pressure, fuel batch), and *what may be concluded* (you can compare two cars tested the same way; you cannot predict your Tuesday). A benchmark result is a sticker figure, and every argument in this booklet is about reading the small print.

The terms, each with its look-alike

TERM	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY — WHAT IT GETS CONFUSED WITH
Benchmarking	Testing performance in a controlled manner , so choices can be compared, regressions found, and limits understood — before they are met in production.	Those limits may be system resources, software limits imposed by a virtualized environment, or limits in the target application itself.	vs a fact about your system. Benchmarks tell you only how fast the system can run the benchmark ; it is up to you to understand the result and work out how it applies to your environment. That sentence is the whole subject in one line.

The reasons	Seven distinct purposes, and the purpose decides what a good result even looks like.	System design — comparing systems or components, often for a purchase decision. Proofs of concept — testing under load before committing. Tuning — finding parameters worth investigating with the real workload. Development — non-regression testing, and driving a product to its limit to find where engineering effort pays. Capacity planning — finding limits directly, or data for modelling. Troubleshooting — verifying a component can still reach maximum performance.	And marketing — determining maximum product performance for promotional use, sometimes called <i>benchmarking</i> . It is on the list because it is real, not because it is illegitimate; but a result produced for it answers a different question from one produced for capacity planning.
The six qualities	What makes a benchmark good: repeatable (so comparisons are possible), observable (so performance can be analysed and understood), portable (so it runs on competitors and across releases), easily presented (so everyone understands the results), realistic (so measurements reflect what customers experience), runnable (so developers can test changes quickly).	When comparing systems with intent to purchase, one more is needed: the price/performance ratio , where price can be quantified as the five-year capital cost of the equipment.	Note that <i>observable</i> is doing more work than it appears. A benchmark you cannot watch while it runs cannot be analysed, and an unanalysable benchmark fails the only test that matters in Part 4.
Benchmark analysis	The four things you must understand about any result: what is being tested; what the limiting factor or factors are; any perturbations that might affect the results; and what conclusions may be drawn.	These require a deep understanding of what the benchmark software does, how the system responds, and how the result relates to the destination environment.	vs collecting the number. Given the tool and access to the system, these needs are best served by performance analysis of the system while the benchmark is running — including drill-down analysis to explain and quantify the limiting factors.
The staffing mistake	Having junior staff execute the benchmarks and then bringing in performance experts to explain the results after the run has completed.	Described as a common mistake, and it is a structural one rather than a personal failing.	The correction is scheduling, not skill: engage the experts during the benchmark, so they can analyse the system while it is still running. Once the run has finished, most of the evidence needed to answer three of the four questions is gone with it.
Custom benchmark software	Writing your own tool, or at least your own load generator, instead of adopting one.	Kept short and focused on only what your test needs.	vs a general tool, which must serve many purposes and is correspondingly larger. The advantage is that a short program is quick to analyse and debug — which matters because you will be analysing it.
Reading someone else's result	The case where you have access to neither the tool nor the system.	Apply the same four questions to whatever material is available.	Plus two that only arise here: what is the system environment, and how is it configured? Without those, the result is a number attached to an unknown machine.

What good analysis actually reads like

A description of an early network stack, written in 1981, is worth reading as a model of the standard being asked for here. Four megabytes were transferred between user processes on two machines, partitioned into 1024-byte records and encapsulated in 1068-byte packets. The transfer took **28 seconds** including connection setup and teardown, giving a user-to-user throughput of **1.2 Megabaud**. During the transfer the sending machine was **CPU saturated** while the receiving machine had about **30% idle time**. The system time was spread across Ethernet handling (20%), IP packet processing (10%), TCP processing (30%), checksumming (25%) and user system-call handling (15%), **with no single part dominating**.

Notice what that paragraph does and does not do. It states the workload precisely enough to reproduce. It identifies the limiting factor — one machine was CPU saturated, so the number is a statement about that machine's processor, not about the network. It then decomposes where the time went, which is what turns a number into an explanation. And it stops: it draws no conclusion the measurement does not support. Being able to break down where kernel CPU time goes at this level only became straightforward much later, with flame graphs; this analysis predates them by decades.

CONCEPT BOUNDARY — A BENCHMARK IS NOT AN EXPERIMENT ABOUT YOUR SYSTEM

What it is: a controlled measurement of how fast this system ran this particular program under these particular conditions.

What it is not: a prediction of your production performance. Getting from one to the other requires you to establish that the workload resembles yours, that the configuration resembles yours, and that the factor that limited the run is one that would also limit you.

The practical consequence: the sentence “this system does X operations per second” is never a complete claim. The complete claim names the workload, the configuration and the limiter — and a result missing all three is not a weak result, it is an uninterpretable one.

Anchor sketch — from a number to a claim

Legend: bold = a named node

reversed = the node that matters most

Q = cover the page and answer this

a benchmark tells you ONE thing:
how fast this system ran THIS BENCHMARK.
getting from there to a claim about your system is
YOUR work, and it is most of the job.

|

WHY ARE YOU RUNNING IT the purpose sets the standard
system design . proof of concept . tuning .
development (non-regression, limit-finding) .
capacity planning . troubleshooting . marketing

|

WHAT MAKES ONE GOOD
repeatable . observable . portable .
easily presented . realistic . runnable
+ price/performance, when you intend to buy
(price = the five-year capital cost)
+--> "observable" is load-bearing: a benchmark you
cannot watch cannot be analysed, and an
unanalysable benchmark fails Part 4 entirely

|

THE FOUR QUESTIONS ask these of every result

- 1 what is being tested?
- 2 what is the limiting factor?
- 3 what perturbations might affect it?
- 4 what conclusions may actually be drawn?

+--> best answered by analysing the system WHILE
the benchmark runs. afterwards, most of the
evidence for three of the four is gone.

+--> if you have no access at all, add: what is the
environment, and how is it configured?

|

the structural mistake: juniors run it, experts explain
it afterwards. by then the machine is idle.

Q Give the four questions, in order, and say which one most results omit.

Q Name the six qualities of a good benchmark, and the seventh needed before a purchase.

Q Why is “bring in the expert afterwards” a structural mistake rather than a staffing one?

Trade-offs

DECISION	BUYS YOU	COSTS YOU	CHOOSE IT WHEN
Benchmarking hardware before purchase	Evidence for a large commitment, including the price/performance ratio.	In on-premises environments, a process that can run weeks or months — shipping, racking, cabling and installing before any testing starts.	Large capital commitments, typically revisited every year or two as new hardware appears.
Benchmarking in an on-demand environment	A large-scale environment created in minutes, used, and destroyed at very little cost.	Instances may land on faster or slower hosts with different contention, so a single run can mislead.	Almost always available now; it is what makes scale testing practical at all.

Testing new instance types with production traffic instead	The real workload, skipping a traditional evaluation entirely.	You lose the controlled comparison, and you need a fault-tolerant environment to risk it.	Where the environment permits it — and benchmarking still helps afterwards, to explain the differences in detail.
Writing your own benchmark or load generator	Short, focused code that is quick to analyse and debug — and it may reveal useful details about the target along the way.	Your time, and the risk of a subtle bug in software nobody else has reviewed.	When the available tool is too large to understand, or tests more than you need.
Using a published industry result	Avoids executing the benchmark yourself; disclosures often include configuration detail.	You must find the one that resembles your workload, and you are trusting someone else's four answers.	Early comparison across many vendors, before narrowing to your own testing.
Engaging the performance expert during the run	All four questions answerable, with drill-down to quantify the limiter.	Coordination — the expert's time has to be booked for the run, not for the readout.	Every benchmark that anyone will act on.

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO <i>UNDERSTAND</i> IT	SAY IT THIS WAY TO SOUND LIKE AN <i>EXPERT</i>
“The system does 50,000 operations a second.”	“It ran <i>that benchmark</i> at 50,000 a second. What was the workload, what was the configuration, and what stopped it going higher?”
“We ran the tests and here are the numbers.”	“What was the limiting factor in each? A result without one means we never established that the system was driven to its limit.”
“Let's get the experts to review the results.”	“Get them on the run, not the readout. Three of the four questions need the system live, and by the readout the machine is idle.”
“This tool is the standard one for this.”	“Can we watch it while it runs? If it isn't observable we can't analyse it, and an unanalysable benchmark isn't a good benchmark.”
“It's the fastest option.”	“At what price? Before a purchase the relevant figure is price/performance, priced as the five-year capital cost.”
“The vendor published a result.”	“What was the system environment and how was it configured? Without those it's a number attached to an unknown machine.”

PART 1 · QUESTION 1

A team is choosing between two managed database offerings. An engineer runs a load tool against both for ten minutes each, produces a slide showing one is 1.8× faster, and circulates it. Asked what limited each run, they say the tool reported no errors and both machines “looked busy”. The slide is scheduled for a purchasing decision on Friday. The team's performance specialist is offered a slot on Thursday to “review the numbers”.

(a) Name the four things that must be understood about any benchmark result, and say which of them this exercise has established. **(b)** Explain why the Thursday review slot is the wrong intervention, in terms of what evidence still exists by then. **(c)** Name the additional quantity that this particular purpose requires, say how it is defined, and give two further questions you would put to the engineer before Friday.

PART 1 · QUESTION 2

Two results land on your desk on the same morning. The first says: “Storage cluster: 1.4 million operations per second.” The second reports a four-hour soak test of an internal message bus. It states the payload size and the batch size, gives the sustained rate and the total volume moved, notes that the producer host's network interface sat at 96% of line rate for the whole run while the consumer host's processors never went above 40%, breaks the producer's processor time down across serialisation, compression, the network stack and the application's own bookkeeping with no single part dominant, and then stops.

(a) Say what the second result does that the first does not, mapping each of its elements to one of the four questions. **(b)** Explain why the observation about the producer's network interface changes what the sustained-rate figure is a claim *about*. **(c)** The first result's author says the second is “overkill for a headline number”. Give the strongest version of their case, then say what you would still require before quoting the 1.4 million figure anywhere.

2x2 — who ran it, and can you still watch it

COLUMNS: CAN YOU OBSERVE THE SYSTEM WHILE THE BENCHMARK RUNS? →

	Yes — you have the tool and the system	No — the run is over, or was never yours
YOU ran it	Everything is available. Use it. All four questions are answerable, and the fourth — what may be concluded — only becomes safe once the first three are. <i>Move:</i> analyse during the run, with drill-down to quantify the limiter. Ideally configure the benchmark to sit in steady state so analysis can continue for hours. As a bonus, this is the best possible practice for your observability tools: a known load, and a chance to see how it appears.	The structural mistake Juniors execute; the expert is booked to explain the numbers afterwards. <i>Move:</i> recognise that this is a scheduling failure, not a skills gap. Once the machine is idle, most of the evidence for three of the four questions is gone. If you are already here, list the assumptions you could not check and publish them <i>with</i> the result — that list becomes the to-do for verification if the result turns out to matter.
SOMEONE ELSE ran it	Rare, and worth taking A vendor proof of concept on hardware you can reach, or a colleague's run still in progress. <i>Move:</i> get on the machine while it is live. This is the one chance to check whether the test does what its documentation claims, rather than arguing about it from a slide afterwards.	You are reading, not measuring A published or vendor result, with no access to either the tool or the system. <i>Move:</i> apply the same four questions to whatever material exists, and add the two that only arise here — what was the system environment, and how was it configured? Treat vendor numbers as an upper bound you should not expect to exceed.

Read it as: the column decides how much of the truth is recoverable, and it closes the moment the run ends. Most benchmarking arguments happen in the right-hand column, about evidence that could have been gathered cheaply in the left-hand one and no longer can.

CARRY-OUT RULE FROM THIS PART

Never quote a benchmark number without its limiter. “X operations per second” is an incomplete claim; “X operations per second, limited by *this*, on *this* configuration, running *this* workload” is a complete one. If nobody can supply the limiter, the run was not driven to its limit — or nobody checked, which for your purposes is the same thing.

The sixteen ways it lies

A catalogue of mistakes, fallacies and misdeeds — most of them ordinary, unglamorous and entirely accidental.

TENTH-GRADER VERSION

- The headline failure: you benchmark A, you actually measure B, and you tell everyone you measured C.
- A popular tool is not a correct tool. “Everyone uses it” is not evidence — it is a named logical fallacy.
- A number on its own, with no account of what limited it, is usually the *start* of an investigation, not the end of one.
- Tests can fail invisibly: every request can error, and the tool will still print a tidy latency figure.
- Some failures aren't errors at all. The number is true, and the story told about it leaves out the price, or the configuration, or the fact that the product was built to win this exact test.
- Deliberately faking numbers is the *rarest* item on the list. Nearly everything here is done by honest people in a hurry.

THE EVERYDAY VERSION

A restaurant's food-hygiene rating. Most of the ways it can mislead you have nothing to do with dishonesty. The inspection happened on one Tuesday in March, when the head chef was in and the kitchen was quiet — that is *ignoring variance*. It graded the kitchen, not the food, so you have inferred something the test never measured — that is *casual benchmarking*. You trust it because everyone does, and you have never read what the grades mean — that is *blind faith*. Meanwhile the restaurant next door hired a consultant who knows exactly which items the inspector scores and has optimised for those specifically — that is a *benchmark special*, and it is legal, and the resulting five is genuine. Only one item on the whole list is forging the certificate, and that is the one that almost never happens.

The sixteen, each with its look-alike

FAILURE	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY — WHAT IT GETS CONFUSED WITH
1 • Casual benchmarking	Treating benchmarking as fire-and-forget. Summarised as: you benchmark A, but actually measure B, and conclude you've measured C.	Many tools claim or imply they measure disk performance but actually test file system performance. The difference can be orders of magnitude , as file systems use caching and buffering to substitute disk I/O with memory I/O.	vs a broken tool. The tool may be functioning correctly and testing the file system exactly as designed — and your conclusions about the disks will still be wildly incorrect. The defect is in the inference, not the software.
2 • Blind faith	Believing a benchmark because it is popular, open source, and long-established. The misconception that popularity equals validity is <i>argumentum ad populum</i> .	A flawed micro-benchmark promoted by a top technology company, known among veteran performance engineers to be flawed and never to be used — with no easy way to stop it spreading.	vs a software bug. The problem is not even necessarily with the benchmark software — though bugs do happen — but with the interpretation of its results. Auditing the code is not sufficient.

3 • Numbers without analysis	Bare results with no analytical detail — this can be a sign that the author is inexperienced and has assumed the number is trustworthy and final. Every benchmark number should be accompanied by a description of the limit encountered and the analysis performed.	Summarised as: if you've spent less than a week studying a benchmark result, it's probably wrong.	vs having no time. If you genuinely don't, the honest move is to list the assumptions you haven't checked and publish them with the result — that the tool isn't buggy, that the disk test measures disk I/O, that the tool drove it to its limit, that this workload is relevant. The list becomes the to-do for verification if the result later matters.
4 • Complex benchmark tools	A tool whose own complexity hinders the analysis. Ideally the program is open source so it can be studied, and short enough to be read and understood quickly.	Prefer C for micro-benchmarks; for client simulation, use the same language as the client to minimise differences. Complex suites make analysis hard through sheer volume of code.	vs the specific failure of benchmarking the benchmark — where the reported result is limited by the benchmark software itself. A common cause is single-threaded benchmark software , which measures one thread's ceiling and reports it as the system's.
5 • Testing the wrong thing 6 • Ignoring the environment	Running an available benchmark rather than a relevant one; and testing on a system that does not resemble production.	Testing disk performance because disk tools exist, when the target workload runs entirely out of file system cache. Or evaluating a database on a default server — default tunables, default file system — when the production servers are tuned for high disk IOPS.	vs a benchmark that <i>used</i> to be right. One may have tested an appropriate workload once and not been updated for years. The related organisational version: a team standardises on a benchmark and spends all its performance effort on it — if it doesn't resemble customer workloads, the engineering optimises the wrong behaviour.
7 • Ignoring errors	A result is produced; that does not mean the test succeeded. Some — or all — requests may have errored.	A web server benchmark reported average latency of over one second . The server had done nothing at all: all requests were blocked by a firewall, and the latency shown was the time for the benchmark client to time out and error.	Covered by the earlier failures, but singled out because it is so common . Note the shape: the failure produced a plausible number in the right units, not an obvious crash.
8 • Ignoring variance	Applying a steady, consistent workload based on the average of real-world measurements, when the real rate and the real mix vary.	A disk workload averaging 500 reads/s and 50 writes/s is simulated at those rates, or at the 10:1 ratio so higher rates can be tested. But writes may arrive in bursts every 10 seconds (asynchronous write-back flushing) while synchronous reads are steady.	vs perturbation. This is variance <i>in the workload you are simulating</i> . Bursts can cause real production issues — by queueing the reads — that a steady average simply never reproduces. One way to simulate it is a Markov model in the benchmark.
9 • Ignoring perturbations	External activity affecting the result: a timed system backup, monitoring agents collecting once per minute, or — in the cloud — unseen tenants on the same host.	The strategy for ironing them out is longer runs — minutes instead of seconds . As a rule, a benchmark should not be shorter than one second . Run it several times and examine the standard deviation ; it should be as small as possible.	Short tests might be unusually perturbed by device interrupts pinning the thread during service routines, by kernel scheduling decisions that wait before migrating queued threads to preserve CPU affinity, and by CPU cache warmth effects. Also collect the distribution of operation latency — not just total runtime — so outliers can be seen and recorded.

10 · Changing multiple factors	Comparing two results without understanding everything that differs between them.	Two hosts benchmarked over the network: is the network identical? Is one more hops away, over a slower or more congested link? In the cloud, instances created for a test may land on faster or slower systems, or ones with more tenant contention.	The countermeasure is not one careful run but several : test multiple instances and take the median — better, record the distribution — to avoid an outlier caused by testing one unusually fast or slow system.
11 · Benchmark paradox	“If your product's chances of winning a benchmark are 50/50, you'll usually lose.”	Customers buying on performance often want to be sure, so they run several benchmarks and want the product to win them all. At 50% each, the probability of winning three is $0.5 \times 0.5 \times 0.5 = 0.125 = 12.5\%$.	vs a reason to ignore benchmarks — ignoring benchmarks is itself a benchmarking pitfall . The paradox is an argument for making your results <i>reliably</i> good, not for opting out. And the odds in practice are worse than the coin-flip they start from.
12 · Benchmarking the competition	Producing results showing your product beats a competitor's. Far more difficult than it sounds.	Customers don't use a product for five minutes; they use it for months, analysing and tuning it. You do not have weeks to tune a competitor, so you can only gather untuned — and therefore unrealistic — results .	The self-defeating part: the competitor's customers, the target of the exercise, may well recognise untuned results, so your company loses credibility with the very people it was trying to impress . If you must do it, tune their product seriously — best practices, customer forums, bug databases, outside expertise — then make the same effort on your own before going head to head.
13 · Friendly fire	Underselling your own product: not testing the top-performing configuration, or not driving the system to its true limit.	An engineering team's high performance depended on a new technology that had not yet been documented. The benchmark team didn't understand it, misconfigured it, and published numbers that undersold the product .	Also counts: benchmarking an older software version with issues already fixed, or on whatever limited equipment happened to be available. Potential customers assume a published company benchmark shows the best possible performance — so not providing that undersells the product. The guard is the question “what is the bottleneck for this benchmark?” , and sharing results with engineering before publication.
14 · Misleading benchmarks	Results that are technically correct but misrepresented — through unintentionally limited information about what was measured, or deliberately omitted information.	A vendor builds a prohibitively expensive custom product, achieves a fantastic result, does not disclose the price, focuses on non-price/performance metrics, and markets an ambiguous summary (“we are 2× faster!”) associated with the company or product line in general.	vs cheating. The numbers are not fake — it is lying by omission. A second variant: a marketing budget engages several third parties, not for expertise but for a fast cheap result, and picks the best of the group; non-expertise is arguably advantageous, since a large deviation in a favourable direction is what is wanted. Treat such results as upper bounds you should not expect to exceed .

15 · Benchmark specials	The vendor studies a popular or industry benchmark and engineers the product to score well on it , disregarding actual customer performance. Also called optimising for the benchmark.	The term dates from 1993 and the TPC-A benchmark: a database vendor was charged with adding a special option purely to inflate its results — something a typical customer wouldn't use — and replied, with some justification, that it had followed the letter of the specification, which said nothing about benchmark specials.	The fix was specification, not argument. An anti-benchmark-special clause now prohibits implementations that improve benchmark results but not real-world performance or pricing, and — since the focus is price/performance — a price requirement that the total price be within 2% of what a customer would pay . These were addressed many years ago and you shouldn't necessarily expect them today .
16 · Cheating	Sharing fake results.	Described as either rare or nonexistent: no case of purely made-up numbers being shared, even in the most bloodthirsty of benchmarking battles .	vs everything above it. This is the failure people imagine when they distrust benchmarks, and it is the least likely one they will meet. Suspecting fraud is usually a way of not doing the analysis that would find the real cause.

CONCEPT BOUNDARY — A WRONG MEASUREMENT IS NOT THE SAME AS A WRONG CLAIM

A wrong measurement means the number does not describe what you think it describes: the disk test hit the cache, every request errored, the tool's single thread was the ceiling, a backup ran through the middle of it. Fix these by analysing the run.

A wrong claim means the number is accurate and the story around it is not: the price is missing, the configuration is not one anybody would buy, the product was built specifically to win this test, or a specific result has been generalised into a slogan. Fix these by asking for what was left out.

Why the distinction earns its place: the two have different remedies and different tells. No amount of careful measurement protects you from a claim error, and no amount of scepticism about marketing protects you from a measurement error in your own lab. Most people are alert to one kind and blind to the other — and it is usually their own work where they are blind.

Anchor sketch — sorting the sixteen

Legend: bold = a named group

reversed = the group that holds almost everything

Q = cover the page and answer this

sixteen failures. sort them by what broke, and by whether anyone meant it.

|

THE MEASUREMENT IS WRONG, BY ACCIDENT (10 of 16)

- 1 casual benchmarking .. measured B, claimed C
- 2 blind faith popular != valid
- 4 complex tools incl. benchmarking the benchmark (single-threaded)
- 5 testing the wrong thing
- 6 ignoring the environment (untuned test box)
- 7 ignoring errors firewall case: latency WAS the client timeout
- 8 ignoring variance bursts, not averages
- 9 ignoring perturbations. run >= 1 second; use minutes; check std dev
- 10 changing multiple factors .. median of several
- 13 friendly fire your own product, undersold

|

THE CLAIM IS WRONG, THE NUMBER ISN'T

- 3 numbers without analysis . no limiter stated
- 11 benchmark paradox $0.5^3 = 12.5\%$
- 12 benchmarking the competition . untuned, so the audience stops believing you
- 14 misleading lying by OMISSION: price withheld, "2x faster" generalised
- 15 benchmark specials TPC-A, 1993. now: anti-special clause + price within 2%

|

DELIBERATE FABRICATION

- 16 cheating rare or nonexistent. the one you fear is the one you won't meet.

Q State the casual-benchmarking formula in its A/B/C form, and give the disk example.

Q Which failure was singled out because it is so common, and what did its worked case actually measure?

Q Why is a 50/50 product usually a losing product, and what is the arithmetic?

Trade-offs

DECISION	BUYS YOU	COSTS YOU	CHOOSE IT WHEN
Publishing a result with the unchecked assumptions listed	An honest result now, plus a ready-made verification to-do list if it turns out to matter.	It looks less authoritative than a bare number, and some readers will discount it for that.	Whenever you do not have time for the analysis — which is most of the time.
Longer runs (minutes, not seconds)	Perturbations average out, and interrupts, scheduling decisions and cache-warmth effects stop being able to dominate a short run.	Wall-clock time, multiplied by every cell of your test matrix.	Always — the floor is one second, and the working target is minutes.

Simulating variance with a Markov model instead of steady average rates	Reproduces bursts, which is where real production issues live.	A more complex benchmark, which then has to be analysed itself.	Where the real workload is bursty and the burst is the thing that hurts — write flushes queueing reads.
Testing many instances and taking the median	Protection against one unusually fast or slow machine.	More runs, more cost, more analysis.	On-demand environments, where the machine underneath you is not a constant.
Benchmarking a competitor at all	Marketing gets the comparison it asked for.	Untuned results, and credibility lost with exactly the audience you wanted.	Only after serious tuning effort on their product <i>and</i> the equivalent on yours.
Using a vendor's published number	A free upper bound: a value you should not expect to exceed.	You inherit their four answers, and possibly a withheld price and an unbuyable configuration.	Early screening — and read the fine print for system, disk types and counts, network interfaces and configuration.

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO UNDERSTAND IT	SAY IT THIS WAY TO SOUND LIKE AN <i>EXPERT</i>
“The disk test came back at 500 MB/s.”	“Did it reach the disks at all? File systems substitute memory I/O for disk I/O, and the difference is orders of magnitude.”
“It's the standard tool, everyone uses it.”	“Popularity isn't validity — that's <i>argumentum ad populum</i> . And the risk here is usually the interpretation, not a bug.”
“Latency was about a second.”	“What was the success/fail ratio? A timeout is also a latency, and it's a plausible-looking one.”
“We modelled it at the average rate.”	“The average erases the bursts, and the bursts are what queue the reads. Can we drive the transitions, not the mean?”
“The run took eight seconds, that's plenty.”	“Push it to minutes and give me the standard deviation across repeats. Short runs get pinned by interrupts and scheduling.”
“We're 2× faster than them.”	“On which configuration, at what price, and against which of their versions — tuned by whom?”
“I think they faked it.”	“Fabrication is the rarest failure on the list. Far more likely the number is real and something was omitted — start with the price and the configuration.”

PART 2 · QUESTION 1

A team evaluates two message brokers. The load tool is run for 20 seconds against each; broker A reports $3\times$ the throughput of broker B and is chosen. Later, in production, A is slower. Investigation finds four things: the tool ran as a single process on one client machine; broker B had been configured to acknowledge writes durably while A had not; roughly 8% of B's requests had returned errors that the tool counted as completed operations; and the nightly index rebuild had been running on the B host for the last six seconds of its run. The engineer's summary slide had said only "A: 240k msg/s, B: 80k msg/s".

(a) Name each of the four named failures at work here, and say for each whether it made the *measurement* wrong or the *claim* wrong. **(b)** Two of these four have a stated remedy that costs almost nothing; give both remedies and the specific rule of thumb attached to one of them. **(c)** The slide itself commits a fifth failure independent of the four. Name it and state what the slide should have carried instead.

PART 2 · QUESTION 2

Your company's product must win a customer evaluation consisting of four separate benchmarks; the customer will buy only if you win all four. Your engineers believe you are genuinely a coin-flip against the incumbent on each. Marketing proposes three responses: (i) commission an outside firm quickly and cheaply to produce a comparison against the incumbent; (ii) add a configuration switch that detects the evaluation's access pattern and pre-fetches for it; (iii) skip benchmarks entirely and sell on features.

(a) Compute your probability of winning all four, and state the summarised rule this illustrates. **(b)** Take each of the three proposals in turn, name the failure it corresponds to, and give the specific reason it backfires — for (ii), say which body introduced a mechanism against it, what that mechanism is, and what price rule accompanies it. **(c)** Name the failure that proposal (iii) itself commits, and describe the one intervention that genuinely improves the odds.

2x2 — what broke, and did anyone mean it

COLUMNS: WAS IT INTENDED? →

	Accidental	Deliberate
THE MEASUREMENT is wrong	Where ten of the sixteen live Casual benchmarking, blind faith, complex or single-threaded tools, testing the wrong thing, ignoring the environment, errors, variance and perturbations, changing multiple factors, friendly fire. <i>Move:</i> analyse the system while the benchmark runs. Nearly every cell entry here is detectable live and undetectable afterwards. This is the quadrant your own work lands in, which is why it is the one to guard.	Cheating — and it barely happens Sharing fake results: rare or nonexistent, with no observed case of purely made-up numbers even in the fiercest benchmarking battles. <i>Move:</i> stop suspecting it. Reaching for fraud as an explanation is usually a way of avoiding the analysis that would find the real cause — which will almost certainly be somewhere in the cell to the left.
THE CLAIM is wrong, the number isn't	Incomplete, not dishonest Numbers published with no analysis and no limiter; the benchmark paradox, and the temptation to give up on benchmarking because of it; untuned competitor results; misleading results caused by unintentionally limited information about what was measured. <i>Move:</i> ask what was left out, and supply it in your own work — the limiter, the analysis, and the assumptions you couldn't check.	Lying by omission Price withheld on an unbuyable custom configuration; “2× faster” generalised to a whole product line; several cheap third parties commissioned and the best result kept; and benchmark specials — engineering the product to score on the test. <i>Move:</i> read the fine print for system, disks, interfaces and configuration; treat the figure as an upper bound. Note that the transaction-processing body answered specials with rules rather than argument: an anti-special clause, and total price within 2% of what a customer would pay.

Read it as: the row tells you which remedy applies — measurement errors are fixed by observing the run, claim errors by demanding what was omitted. The column tells you where to look first: thirteen of the sixteen sit on the left. One failure, **misleading results**, appears in both columns and is meant to — it can come from information that was unintentionally limited or from information that was deliberately withheld, and the number is correct either way.

CARRY-OUT RULE FROM THIS PART

Before you doubt anyone's honesty, doubt the measurement. When a result surprises you, the ordered list of suspects is: the test measured something else; the run errored; the tool was the bottleneck; something perturbed it; the configurations differed. Fabrication is last, and in practice never. This ordering matters because the first five are fixable by you and by this afternoon, and the last one is only an argument.

Choosing the kind

Micro-benchmark, simulation, replay, industry standard — what each one can answer, and what it structurally cannot.

TENTH-GRADER VERSION

- There is a spectrum of benchmark types, arranged by how much they resemble the real workload — with the real production workload sitting at one end of it.
- A **micro-benchmark** tests one narrow operation. Easy to understand, easy to root-cause, and obviously artificial — which is a feature, because nobody mistakes it for reality.
- A **simulation** imitates a real customer workload from measurements of it. Closer to reality, harder to analyse.
- A **replay** plays a recorded trace back at the system. Sounds ideal. Usually isn't.
- An **industry-standard** benchmark is one an independent body defines, documents and audits, so results across vendors can be compared at all.
- Designing micro-benchmarks means multiplying: the tests you want, times how much data, times how many threads. And after you have tested the good weather, test the bad.

THE EVERYDAY VERSION

Ways to find out whether someone can drive. A **micro-benchmark** is the reverse-parking exercise on an empty test pad: one skill, isolated, repeatable, and obviously not a journey — you would never mistake the score for “can drive”. A **simulation** is the driving test, built from a survey of what real driving contains: 40% urban, 19% junctions, 7% motorway, and the mix is chosen deliberately. A **replay** is putting the candidate in a simulator that plays back the exact route somebody else drove last Tuesday — the trouble is that when the candidate behaves differently, the recording carries on regardless, because the traffic in it was never going to react to them. And the **industry standard** is the national licence: defined by an independent body, run to published rules, and comparable between one candidate and another precisely because nobody gets to design their own exam.

The four kinds, each with its look-alike

KIND	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY — WHAT IT GETS CONFUSED WITH
Micro-benchmark	An artificial workload testing one particular type of operation — a single kind of file system I/O, database query, CPU instruction or system call.	The advantage is simplicity: narrowing the components and code paths gives an easier target to study and lets differences be root-caused quickly. Tests are usually repeatable, because variation from other components is factored out; and they are quick to run across different systems.	vs a simulation. Because they are deliberately artificial, micro-benchmarks are not easily confused with real workload simulations — the artificiality is protective. The cost: results must be mapped to the target workload , and of the several dimensions one may test, only one or two may be relevant.

Designing a micro-benchmark set	Choosing the attributes to vary, then multiplying them out. For a file system: sequential or random, I/O size, and direction.	Five sample tests, each with a stated intent: sequential 512-byte reads (maximum realistic IOPS); sequential 1 MB reads (maximum read throughput); sequential 1 MB writes (maximum write throughput); random 512-byte reads (the effect of random I/O); random 512-byte writes (the effect of rewrites).	Every test is then multiplied by two more factors. Working set size: much smaller than main memory, so data caches entirely and the file system <i>software</i> is what you investigate; or much larger, to minimise cache and drive toward <i>disk</i> I/O. Thread count (assuming a small working set): single-threaded, testing file system performance at the current CPU clock speed; or enough threads to saturate all CPUs, testing the maximum of system, file system and CPUs together.
Sunny day vs cloudy day testing	Benchmarks that focus on top speeds are called sunny day performance testing .	The matrix above multiplies quickly; statistical analysis techniques can reduce the set that actually needs running.	So that issues are not overlooked, also consider cloudy day or rainy day performance testing — non-ideal situations, including contention, perturbations and workload variance. Note this is the same list that Part 2 catalogued as failures; here they are deliberately induced.
Simulation (macro-benchmark)	Imitating a customer application workload, often based on workload characterization of the production environment to decide what to simulate.	A production file-server workload found to be composed of reads 40%, writes 7%, getattr 19%, readdir 1%, and so on. Other characteristics can be measured and simulated too.	vs a micro-benchmark: a simulation can encompass many factors that would be time-consuming to investigate one at a time, and can include the effects of complex system interactions that micro-benchmarks may miss altogether . It can produce results that resemble how clients will perform with the real workload — if not closely, at least closely enough to be useful.
Stateless vs stateful simulation	Stateless: each server request is unrelated to the previous one — a series of operations, each type chosen randomly according to the measured probability.	Stateful: each request depends on client state, at minimum the previous request. Reads and writes may arrive in groups, so the probability of a write following a write is much higher than a write following a read.	Such a workload is better simulated with a Markov model , representing requests as states and measuring the probability of state transitions. This is the same instrument Part 2 named as the answer to ignoring variance — and it is needed here for the same reason.
Ageing of simulations	Customer usage patterns change over time, so simulations need updating and adjusting to stay relevant.	Two well-known CPU benchmarks are simulations of their era: one developed in 1972 to simulate the scientific workloads of the time, another from 1984 simulating the integer-based workloads of the time.	There may be resistance to updating, because published results based on the older version would no longer be comparable with the new one. Note the trap: the property that makes a benchmark valuable — comparability across time — is the same property that keeps an obsolete one alive.

Replay	Capturing a trace log and replaying it at the target, testing performance with the actual captured client operations. Sounds ideal — as good as testing in production.	It is problematic: when characteristics and delivered latency change on the server, the captured client workload is unlikely to respond naturally to those differences , which may prove no better than a simulated workload. When too much faith is placed in it, things can get worse.	The worked case: a storage upgrade evaluated by replaying the production trace on new hardware. Performance was worse and the sale was lost. The replay operated at the disk I/O level; the old system had 10K rpm disks and the new one slower 7,200 rpm disks — but 16 times the file system cache and faster processors. The real workload would have improved, returning largely from cache, which replaying disk events never simulated.
Industry-standard benchmarks	Benchmarks from independent organisations aiming to be fair and relevant — usually a collection of micro-benchmarks and workload simulations, well defined, documented, and executed under guidelines so results are as intended.	Vendors may participate, usually for a fee, which provides the software to execute the benchmark. A result usually requires full disclosure of the configured environment, which may be audited.	vs running your own. For the customer these save a lot of time, since results may already exist for many vendors and products. Your task becomes finding the benchmark that most closely resembles your future or current production workload — and for a current one, that is decided by workload characterization.
The two bodies, and the common metrics	The Transaction Processing Performance Council (TPC) creates and administers industry benchmarks focused on database performance.	TPC-C, a complete computing environment where a population of users executes transactions; TPC-DS, a decision support system with queries and data maintenance; TPC-E, an OLTP workload modelling a brokerage firm's database; TPC-H, decision support with ad hoc queries and concurrent modifications; TPC-VMS, allowing other benchmarks to be gathered for virtualized databases; TPCx-HS, a big data benchmark; TPCx-V, database workloads in virtual machines. TPC results include price/performance.	The Standard Performance Evaluation Corporation (SPEC) publishes a standardized set: SPEC Cloud IaaS 2018 (provisioning, compute, storage and network, using multiple multi-instance workloads); SPEC CPU 2017 (compute-intensive, integer and floating point, with an optional energy metric); SPECjEnterprise 2018 Web Profile (full-system performance for enterprise Java application servers, databases and supporting infrastructure); SPECsfs2014 (a client file access workload for file servers); SPECvirt_sc2013 (end-to-end performance of virtualized hardware, platform, guest OS and applications). SPEC results include how systems were tuned and a component list, but not usually their price.
Cross-vendor metrics	Single figures intended to be comparable across products, in the way miles per gallon is for cars.	The need was set out in a 1985 paper, “A Measure of Transaction Processing Power”, which described the need to measure the price/performance ratio , detailed three benchmarks vendors could execute — Sort, Scan and DebitCredit — and proposed an industry-standard transactions per second (TPS) measure based on DebitCredit. That work later encouraged the creation of TPC.	Others used for the same role: MIPS , millions of instructions per second — a measure of performance, but the work performed depends on the type of instruction, which can be difficult to compare between processor architectures; and FLOPS , floating-point operations per second, the same role for floating-point-heavy workloads. Industry benchmarks typically measure a custom metric that serves only for comparison with itself.

CONCEPT BOUNDARY — REALISM AND ANALYSABILITY TRADE AGAINST EACH OTHER

The spectrum: benchmark types line up by how much they resemble the real workload, with the production workload itself at one end. Micro-benchmarks sit at the artificial end; simulations in the middle; replay closest to production.

What moving toward realism buys: results that resemble what clients will actually experience, including complex system interactions a narrow test would miss entirely.

What it costs: analysability. A micro-benchmark involves few components and few code paths, so a difference can be root-caused quickly and the test is repeatable because other variation is factored out. A realistic workload is realistic precisely because it does not factor anything out.

The trap at the far end: replay looks like it escapes the trade-off — real operations, real system — but a recording cannot react. Change the server's delivered latency and the captured client keeps issuing what it issued before. It buys realism of *input* while losing realism of *behaviour*, which is why it can be worse than an honest simulation.

Anchor sketch — the spectrum, and the matrix underneath it

Legend: bold = a named kind reversed = the kind whose artificiality is a feature Q = cover the page and answer this

```
ARTIFICIAL <-----> REAL
easy to analyse, easy to      hard to analyse, includes
root-cause, obviously        interactions, mistakable
not reality                  for reality

|                             |
MICRO-BENCHMARK  SIMULATION  REPLAY  the production
                  (macro-)      workload itself

|                             |
|                             |    +-- a recording cannot REACT.
|                             |    change the server and the trace
|                             |    issues what it always issued.
|                             |    case: disk-level replay, 10K ->
|                             |    7,200 rpm but 16x cache. sale lost.
|                             |
|                +-- built FROM workload characterization
|                stateless: each request independent
|                stateful : depends on the previous, so
|                use a Markov model
|                and it AGES: the 1972 and 1984 CPU
|                benchmarks simulate THOSE eras
+-- design = tests x working set x threads
    5 tests: seq 512B read / seq 1M read / seq 1M
              write / rand 512B read / rand 512B write
    working set << RAM -> tests the FILE SYSTEM
                >> RAM -> drives it TOWARD disk
    threads    single -> fs at this clock speed
                saturate -> system max
    then: sunny day (top speed) AND cloudy day
          (contention, perturbation, variance)
          |
INDUSTRY STANDARDS  someone else already ran it
independent bodies . defined, documented, guideline-bound
vendors MAY take part, usually for a fee . usually full
disclosure of the environment, and it may be audited
your job shrinks to: WHICH one resembles my workload?
(workload characterization answers that)
```

Q Give the two multipliers applied to every micro-benchmark test, and what each setting of each one tests.

Q Why is a micro-benchmark's artificiality described as an advantage?

Q In the replay case study, what did the trace level cause the test to miss, and which Part 2 failure is it an instance of?

Trade-offs

DECISION	BUYS YOU	COSTS YOU	CHOOSE IT WHEN
Micro-benchmark	Few components and code paths, so differences root-cause quickly; repeatable; quick to run across systems; and unmistakable for reality.	Results mean nothing until mapped to the target workload, and only one or two of the dimensions tested may be relevant.	You want to understand a component, compare systems quickly, or find where an optimisation actually lands.
Working set much larger than main memory	Minimises the file system cache, driving the benchmark toward testing disk I/O.	More data to stage, longer runs, and a slower test overall.	The question is genuinely about the disks — otherwise you are testing the cache and calling it a disk.
Saturating thread count	The maximum performance of system, file system and CPUs together.	You lose the single-threaded reading, which is the one that tracks CPU clock speed.	You want the ceiling; run single-threaded too when you want to attribute it.
Simulation over micro-benchmark	Resembles the real client experience; captures complex interactions a narrow test misses.	Harder to analyse; can ignore variance; needs updating as customer patterns change.	You need a defensible answer about <i>the workload</i> , not about a component.
Keeping an old simulation version	Comparability with results already published.	Relevance — it simulates the workload of the year it was written.	Only while the comparison is worth more than the realism; say which you chose.
Replay over simulation	Actual captured client operations rather than a synthesised mix.	The trace cannot respond to a changed server, and at the wrong level it can invert the answer entirely.	Rarely — and only when the trace level matches the level the change affects.
Industry-standard result	Saves running anything; disclosures cover configuration and tuning, and may be audited.	Fit is approximate, a fee is usually involved to participate, and price is disclosed by some bodies and not others.	Comparing vendors and products before committing to your own testing.

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO UNDERSTAND IT	SAY IT THIS WAY TO SOUND LIKE AN EXPERT
“It's a synthetic test, so it's not very useful.”	“It's a micro-benchmark. Being obviously artificial is the point — nobody mistakes it for the workload, and it root-causes fast.”
“We ran the disk test on a small file.”	“Working set well under main memory tests the file system software, not the disks. Which question were we answering?”
“We built a workload that looks like production.”	“Characterized from what, and is it stateless or stateful? If writes group, you need transition probabilities, not a flat mix.”
“We'll just replay last week's traffic.”	“At which level? A trace can't react to a faster server, and a disk-level replay hides a cache improvement completely.”
“We always test peak throughput.”	“That's sunny day testing. Schedule the cloudy day runs too — contention, perturbation, variance.”

“There's a standard benchmark for this.”	“Which one resembles our workload? That's a workload-characterization question, and the disclosure will tell us how they tuned it.”
“Their number is in a different unit to ours.”	“Most industry benchmarks report a custom metric that only compares with itself. Cross-product figures like instructions or floating-point operations per second have their own comparability problems.”

PART 3 · QUESTION 1

A team is replacing an object storage tier. The new tier has slower spinning media but four times the memory for metadata and small-object caching. They plan to capture a week of production traffic at the block device layer and replay it against the new tier, on the grounds that this is “as close to production as we can get without risking production”. A colleague suggests instead measuring the production request mix and driving a synthetic workload from it, and separately running a small set of narrow tests.

(a) Predict the result of the replay plan and explain the mechanism, naming the level at which the trace was captured and why that matters here. **(b)** Name the general property of replay that makes it disappointing even when the capture level is right. **(c)** For the colleague's narrow tests, specify the two multiplying factors that must be applied to each test, and say what setting of each you would use to answer the question “does the extra memory help?”

PART 3 · QUESTION 2

Your team maintains an internal benchmark written six years ago to imitate the then-current API traffic of your service. Traffic has since shifted heavily toward a batch endpoint that did not exist when the benchmark was written. A proposal to rewrite it is resisted: two years of published quarterly results use the current version, and rewriting would break the trend line. Separately, a vendor offers you results from an independent body's suite instead.

(a) Name the kind of benchmark your internal one is, and the specific weakness of that kind that this situation is an instance of. **(b)** State plainly what is being traded against what in the resistance to rewriting, and say what you would do with the two years of results either way. **(c)** Give three things an independent body's result offers that your internal benchmark does not, and the one question you must answer before the vendor's result is usable at all — including the method that answers it.

2x2 — the two multipliers on every micro-benchmark

COLUMNS: HOW MANY THREADS? →

	Single-threaded	Enough threads to saturate all CPUs
WORKING SET much smaller than main memory	You are testing the file system software Data caches entirely, so what you measure is the software path — and single-threaded, it tracks the current CPU clock speed. <i>Move:</i> know that this is the cell people land in by accident when they believe they are testing disks. It is a perfectly good cell to be in deliberately; it is the wrong one to be in without noticing. If your question is about the storage device, you are in the wrong row.	The maximum of the whole stack Still cached, but now driving system, file system and CPUs together to their combined ceiling. <i>Move:</i> use this for the headline number, and pair it with the single-threaded run so you can attribute the difference. On its own it tells you the ceiling without telling you whose ceiling it is.
WORKING SET much larger than main memory	Disk I/O, one request at a time Cache effects are minimised, so the test drives toward disk I/O — but with a single thread there is no queue depth to speak of. <i>Move:</i> good for per-operation latency against the device; poor for the device's throughput ceiling, which needs concurrency.	Disk I/O, driven hard Minimal cache, maximum concurrency: the closest a micro-benchmark gets to a storage-device limit. <i>Move:</i> this is where the disk numbers people quote should have come from. Expect it to be the slowest and most expensive cell to run — large data to stage, and long runs.

Read it as: the row decides *which component* you are measuring, and the column decides *whose limit* you find. Every one of the five sample tests — sequential 512-byte reads, sequential 1 MB reads and writes, random 512-byte reads and writes — multiplies out across all four cells, which is how a short list of tests becomes a large matrix — and statistical analysis techniques can be used to cut the required set back down.

CARRY-OUT RULE FROM THIS PART

Pick the kind from the question, not from the tool you have. If the question is “why is this component slow?”, a micro-benchmark answers it and a realistic workload obscures it. If the question is “what will customers experience?”, a simulation answers it and a micro-benchmark cannot. If the question is “should we buy A or B?”, someone has probably already run a defined benchmark whose environment is usually fully disclosed and may have been audited, and your remaining job is to work out which of theirs resembles you.

Analyse it while it runs

One habit prevents or exposes most of Part 2 — and the methods, checklists and questions that hang off it.

TENTH-GRADER VERSION

- The bad way: start the test, walk away, put the numbers on a slide, hand the slide to management. That is a named anti-method, not a straw man.
- The good way: watch the machine while the test is running, with ordinary observability tools, and check that it is doing what the tool's name claims.
- Doing this once — on a well-known drive benchmark — showed that the test never touched a disk. It was writing *one byte at a time* into memory.
- Keep asking “why not double?” If you can't say what stopped it going twice as fast, you have not found the limit yet.
- Do the arithmetic. If the claimed result would need more bandwidth than the cable has, the result is wrong and no further debate is required.
- And when you push load higher, watch the latency. A bigger number at an unacceptable latency is not a bigger number.

THE EVERYDAY VERSION

Two ways to check a claim that a lorry can carry ten tonnes. The first: send it away loaded, wait for it to come back, and read the ticket it brings you. The second: stand on the weighbridge while it drives on. The second way is not more rigorous in principle — it is just the only one where you can see that the trailer was half empty, that the load was polystyrene, that the driver went round the block instead of up the hill, and that the weighbridge itself tops out at eight tonnes. All of that is visible for exactly as long as the lorry is on the bridge, and none of it is recoverable from the ticket. Active benchmarking is standing on the weighbridge; passive benchmarking is reading the ticket.

The methods, each with its look-alike

METHOD	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY — WHAT IT GETS CONFUSED WITH
Passive benchmarking (the anti-method)	Fire-and-forget: the benchmark is executed and then ignored until it completes. The main objective is the collection of benchmark data . This is how benchmarks are commonly executed.	Its four steps: 1. Pick a benchmark tool. 2. Run it with a variety of options. 3. Make a slide deck of the results. 4. Hand the slides to management.	It is described as an anti-methodology for comparison with active benchmarking , not as a caricature. Results may be invalid from software bugs, limited by the benchmark software, limited by an unrelated component, limited by configuration, subject to perturbations, or benchmarking the wrong thing entirely. Easy to perform, prone to errors: from a vendor it creates false alarms that waste engineering or lose sales; from a customer it produces poor product choices that haunt the company later.

Active benchmarking	Analysing performance while the benchmark is running — not just after it's done — using observability tools.	It lets you confirm that the benchmark tests what it says it tests, and that you understand what that is. It can identify the true limiters of the system under test, or of the benchmark itself — and including those details when sharing results is very helpful.	vs analysing the results afterwards, which is a different method entirely (statistical analysis, below). Ideally the benchmark is configured to run in steady state , so analysis can continue over hours or days. As a bonus, it is good practice with your observability tools: a known load, and a chance to see how it appears through them.
The analysis case study	A well-known micro-benchmark suite whose documentation describes it as a program to test hard drive performance. Its first test scored 739 Kbytes/sec in a per-character sequential-output column.	Sanity check first: if that really were I/O per character, the system would be achieving 739,000 I/O per second — implausible for disks. Disk statistics during the run reported no disk I/O . Block-layer tracing showed no I/O issued and none completed, but buffers dirtied: 808,225 dirty-buffer and 1,025,678 touch-buffer events. Cache statistics showed a pure workload of dirtyies — hundreds of thousands per second, no misses.	Drilling further up the stack: at the virtual file system layer the process had made 1,176,936 write calls against 13 reads and 7 opens, and the size histogram put 668,839 of them at 1 byte . Conclusion: the first test is a 1-byte file system write that buffers in the file system cache , and does not test disk I/O as the description implies. Its no-write-buffering option turned out not to synchronise after every write, and its direct-I/O option failed — so there was no way to do a per-character disk write test at all.
What the case study is not	Not an argument that the tool is unusually bad — it has served people well on many occasions, and was chosen because it is well known and previously studied, with its most suspicious test deliberately picked.	Some would argue it isn't broken: “sequential output” and “per character” never promised disk I/O. For a benchmark that claims to test hard drive performance, that is at least misleading .	A second lesson hides in its history. Older versions let the C library buffer writes before they reached the file system, so the observed write size was 4 Kbytes or more depending on the library and OS version — making cross-OS comparison meaningless. Fixing it created a new problem: new results cannot be compared with old ones .
CPU profiling	Profiling both the benchmark target and the benchmark software, to quickly check what all the software is doing and see if anything interesting shows up. Usually performed as part of an active benchmarking investigation.	A disk micro-benchmark on a proposed new system gave disappointing throughput; the expectation was that the disks or the controller were inferior and needed upgrading. The resource sweep showed the disks were not very busy , with some CPU usage in the kernel.	vs only profiling what you expect to matter. For a disk benchmark you would not expect CPUs to be interesting — the check was made anyway, and 62.17% of CPU samples included a function that was plainly a file system I/O throttle. A resource control was artificially throttling the benchmark: a default setting on the new system but not the old one , overlooked when the benchmark was run. Reading the code was unnecessary; the function name was enough.
Resource sweep during the run	Applying the utilization / saturation / errors check to every resource while the benchmark runs.	Its purpose here is narrow and decisive: to ensure that a limit is found .	The disjunction is the useful part. Either some component, hardware or software, has reached 100% utilization — or you are not driving the system to its limit . There is no third outcome, and “the numbers looked busy” is not one of the two.
Workload characterization during benchmarking	Characterizing the production workload for comparison, to determine how well a given benchmark relates to your current production environment .	It is the method that answers “which industry-standard benchmark resembles us?” and “is this simulation still relevant?”	vs analysing the benchmark. This one analyses <i>production</i> , and uses the result to judge the benchmark. It is the only method here that does not require the benchmark to be running.

Custom benchmarks	Coding the software yourself for simple benchmarks. Keep the program as short as possible , to avoid complexity that hinders analysis.	C is usually a good choice for micro-benchmarks as it maps closely to what is executed — but think about compiler optimisations: the compiler may elide simple benchmark routines if it thinks the output is unused . It may be worth disassembling the binary to see what will actually run.	vs any language with a virtual machine, asynchronous garbage collection or dynamic runtime compilation, which are much harder to debug and control with reliable precision — though you may need them anyway to simulate client software written in them. Always check with other tools while it runs to confirm its operation. A side benefit: writing one can reveal API options for improving performance that production was built too early to use.
Ramping load	Determining maximum throughput by adding load in small increments and measuring delivered throughput until a limit is reached. Graph the results to get a scalability profile , studied visually or with scalability models.	A file system and server scaled with threads, each performing 8 Kbyte random reads on a cached file, added one by one. The system peaked at almost half a million reads per second , checked at the virtual file system layer, which confirmed the 8 Kbyte size and over 3.5 Gbytes/s transferred at peak. The load generator was short enough to print in full, and read directly through the system call to avoid buffering. The limit turned out to be CPU on the server , which prompted a further investigation.	vs pushing until the number stops rising. Measure latency as well as throughput, especially the distribution . Near the limit, queueing delays make latency climb; push too far and the result is no longer reasonable to consider valid. The worked judgement: 990,000 IOPS at 5 ms average, or scraping past 1 million at over 50 ms average with everything heavily queued. The answer you give marketing is 990,000 . Ask whether the delivered latency would be acceptable to a customer.
Sanity check	Checking a result by investigating whether any characteristic doesn't make sense — in particular, whether it would have required a component to exceed a known limit: network bandwidth, controller bandwidth, interconnect bandwidth, or disk IOPS.	A file server benchmarked with 8 Kbyte reads reported 50,000 IOPS on a single 1 Gbit/s interface. $50,000 \times 8 \text{ Kbytes} = \mathbf{400,000 \text{ Kbytes/s}}$, plus protocol headers — over 3.2 Gbit/s , well in excess of the known limit. Results like this usually mean the benchmark tested client caching and never drove the whole workload to the server.	Real single-direction results collected over one 1 Gbit/s interface: 120 Mbyte/s (0.96 Gbit/s) may be fine, as such an interface should reach around 119 Mbyte/s in practice; 200 Mbyte/s (1.6 Gbit/s) is possible only if traffic in both directions was summed, which these were not; and 350, 800 and 1,150 Mbyte/s (2.8, 6.4 and 9.2 Gbit/s) are clearly bogus. In most cases this exercise ultimately discovers the result is bogus. With access to the system you can go further and follow the scientific method: the question under test becomes <i>whether the benchmark result is valid</i> .
Statistical analysis	Three phases: 1. selection of the tool, its configuration, and the system metrics to capture; 2. execution, collecting a large dataset of results and metrics; 3. interpretation with statistical analysis, producing a report.	Used where access to a large-scale system is time-limited and expensive — one “max config” system, and many teams wanting it: sales for proofs of concept, marketing for the best numbers, support for pathologies that appear only at that scale, engineering for new features, quality for non-regression testing and certification. Each has limited time on the machine and much more time afterwards.	vs active benchmarking, which analyses the system <i>while</i> the benchmark runs; and vs passive benchmarking, in which no analysis is performed at all . Because collection is expensive, make an extra effort to ensure metrics are reliable, and collect more statistical properties — variation, full distributions, error margins — so problems surface sooner. For code changes and non-regression testing, understanding variation and error margins is crucial to making sense of a pair of results. Collect as much performance data as the overhead allows, so forensic analysis is possible afterwards.

The benchmarking checklist	Six questions to seek answers to from any benchmark, to verify its accuracy.	Why not double? — why was the rate not double the result? This is really asking what the limiter is , and answering it solves many benchmarking problems, when you discover the limiter is not the intended target of the test. Did it break limits? — the sanity check. Did it error? — errors perform differently from normal operations, and a high error rate skews results.	Does it reproduce? — how consistent are the results? Does it matter? — the workload tested may not be relevant to your production needs; some micro-benchmarks test individual system and library calls your application may not even use. Did it even happen? — the firewall case from Part 2, where a blocked benchmark reported timeout-based latency as its result.
Questions for a result you cannot analyse	A longer list, for when a vendor gives you a result and you have no access to the running benchmark. The goal: determine what is really being measured , and how realistic or repeatable it is.	In general: does it relate to my production workload; what was the configuration of the system under test; a single system or a cluster; what is its cost; what was the client configuration; what was the duration and how many results; is it an average or a peak, and what is the average; what are the distribution details; what was the limiting factor ; what was the success/fail ratio; what were the operation attributes and how were they chosen; does it simulate variance or an average workload; was it confirmed with other analysis tools — provide screenshots ; can an error margin be expressed; is it reproducible.	Then the resource-specific lists. CPU and memory : which processors, overclocked, custom cooling, how many memory modules and how attached, any CPUs disabled, system-wide CPU utilization (<i>lightly loaded systems can perform faster due to higher turbo boosting</i>), cores or hyperthreads, how much and what type of main memory, custom firmware settings. Storage : device configuration including count, type, protocol, RAID, cache size and write-back or write-through; file system configuration and tuning; the working set size ; to what degree and where it cached; how many files. Network : interface count, type and configuration; topology; protocols and socket options; stack tunables. For industry benchmarks, many of these are answered by the disclosure.

CONCEPT BOUNDARY — THREE THINGS PEOPLE CALL “ANALYSING THE BENCHMARK”

Passive: no analysis at all. The objective is data collection; the analysis step does not exist.

Active: analysis of **the system, while the benchmark runs**. This is the only one that can establish what was actually measured, because that evidence exists only while the load is applied.

Statistical: analysis of **the results, after the run**. Legitimate and necessary — it is how you handle expensive, contended, time-limited hardware — but it answers different questions: consistency, variation, error margins, distributions.

Why the distinction matters: statistical analysis can tell you a result is *reproducible*. It cannot tell you the result was of the right thing. A benchmark that measures the file system cache instead of the disk will reproduce beautifully, with a tiny standard deviation, forever.

Anchor sketch — the habit, and everything hanging off it

Legend: bold = a named method or list

reversed = the habit the rest depends on

Q = cover the page and answer this

PASSIVE (the anti-method) 1 pick a tool
 2 run it with options
 3 make a slide deck
 4 hand it to management

|
the alternative

|
ACTIVE BENCHMARKING - analyse the SYSTEM while the
benchmark RUNS. confirms what is really being tested,
and finds the true limiter of system OR benchmark.
run in steady state so analysis can go on for hours.

|
+---- CASE STUDY: a "hard drive" benchmark
| 739 KB/s per CHARACTER would mean 739,000
| IOPS -> implausible. so walk down the stack:
| disk stats > block layer > cache > VFS.
| verdict: 1,176,936 writes, 668,839 of them
| 1 BYTE, into the file system cache.
| no disk involved at all.
|
+---- CPU PROFILING . disks not busy, kernel CPU
| present -> check anyway -> 62.17% of samples
| in an I/O THROTTLE, a default on the new box.
|
+---- RESOURCE SWEEP (utilization/saturation/errors)
| ensures a limit is FOUND. either something
| hit 100%, or you are not driving it hard
| enough. there is no third answer.
|
+---- WORKLOAD CHARACTERIZATION . analyses
| PRODUCTION, to judge whether the benchmark
| resembles it at all.
|
+---- CUSTOM BENCHMARKS . short, so it can be
| analysed. watch for the compiler deleting
| work it thinks is unused.
|
+---- RAMPING LOAD . add load in small increments,
| graph the scalability profile.
| measure LATENCY too:
| 990,000 IOPS @ 5 ms <- give them this
| 1,000,000+ IOPS @ 50 ms <- not this
|
+---- SANITY CHECK . arithmetic beats argument.
| 50,000 x 8 KB = 400,000 KB/s = >3.2 Gb/s on a
| 1 Gb/s link (real ceiling ~119 MB/s). usually
| means the CLIENT cached it.
|
+---- STATISTICAL ANALYSIS . the RESULTS, afterwards.
| for expensive contended hardware. tells you
| if it REPRODUCES - never if it was right.

THE SIX-QUESTION CHECKLIST

why not double? (= what is the limiter?)

did it break limits? did it error?
does it reproduce? does it matter?
did it even happen?

- Q Give the four steps of passive benchmarking, and the single sentence that defines active benchmarking.
- Q In the case study, name the layers checked in order and what each one ruled out.
- Q Which checklist question is really a question about the limiter, and why does answering it solve so much?

Trade-offs

DECISION	BUYS YOU	COSTS YOU	CHOOSE IT WHEN
Analysing during the run rather than after	The only window in which “what was actually measured” is answerable, plus the true limiter of system or benchmark.	An expert's attention has to be booked for the run itself, and the run has to be long enough to observe.	Any benchmark anyone will act on.
Configuring the benchmark for steady state	Analysis can continue for hours or days, and your observability tools get exercised against a known load.	The run no longer resembles a short burst, and it occupies the hardware for longer.	Whenever the benchmark can be made to hold a stable load.
Profiling something you don't expect to matter	Quick discoveries — a throttle, a default, a component nobody suspected — and it narrows study to the software actually in play.	A few minutes on a hypothesis you expect to fail.	When there is any unexplained usage at all; the disk case turned on a small amount of kernel CPU.
Writing your own load generator	Short enough to read and analyse; may reveal target details you can use later.	Your time, plus the compiler and language pitfalls you now own.	The available tool is too big to understand or does more than you need.
Pushing load past the knee	A larger headline throughput figure.	Queueing delay, so latency may become high enough that the result is no longer reasonable to consider valid.	Never for a published figure — report the last point whose latency a customer would accept.
Statistical analysis of results after the fact	Reproducibility, variation, error margins and distributions; the only workable approach on contended max-config hardware.	It cannot establish what was measured, and collection is expensive enough that mistakes mean re-running.	Time on the system is scarce and time afterwards is not — and collect everything you can while you are on it.

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO UNDERSTAND IT	SAY IT THIS WAY TO SOUND LIKE AN <i>EXPERT</i>
“The run finished, let's look at the numbers.”	“Restart it and leave it in steady state. Three of the four questions need the load applied, and we've just thrown that away.”
“The tool says it tests hard drives.”	“Then we should see block I/O issued. Let's check the disk stats, the block layer, the cache and the file system layer before we believe the label.”
“The disks weren't busy, so it's not a disk problem.”	“Right — so profile what <i>was</i> busy. There was kernel CPU time, and that's exactly how a throttle shows up.”

“Everything looked fine during the test.”	“Which resource hit 100%? If none did, we didn't drive it to its limit, and the number is a floor rather than a ceiling.”
“We got it over a million.”	“At what latency? If it's fifty milliseconds and heavily queued, the honest figure is the last one a customer would accept.”
“That result seems high.”	“Do the sum. Operations times size against the interface bandwidth — if it exceeds the link, it's the client's cache, not the server.”
“We ran it five times, it's consistent.”	“Consistency tells us it reproduces, not that it's measuring the right thing. A cached test reproduces perfectly.”
“Their result is 2× ours.”	“What was the limiting factor, the success/fail ratio, the duration, and is that an average or a peak? And can we have the screenshots from their analysis tools?”

PART 4 · QUESTION 1

A colleague reports that a new caching layer sustains **400,000 reads per second** of 4 Kbyte objects, measured from a bank of client machines across a single 10 Gbit/s network interface into the cache server. They add that the run lasted 30 seconds, the client CPUs were at 40%, the server CPUs were at 55%, and no component reached saturation. They would like to publish the figure.

(a) Perform the check that this result invites, showing the arithmetic, and state your conclusion. **(b)** Give the most likely explanation for a result of this shape, and name the check that would have caught it during the run. **(c)** Independently of the arithmetic, the sentence “no component reached saturation” is itself a finding. Say what it establishes about the figure, using the disjunction that the resource sweep provides, and give the checklist question that puts the same point in three words.

PART 4 · QUESTION 2

You have four hours of exclusive access to a shared, heavily contended reference system next month; five other teams want the same slot. Your task is to establish whether a proposed configuration change improves throughput, and to produce a figure that survives scrutiny. You are told the change “should be about 15% — run it before and after and take the difference”.

(a) Name the analysis approach this access pattern forces you into, give its three phases, and say precisely what it can and cannot establish. (b) Explain why a single before-and-after pair is insufficient for a change of this size, naming the two properties you must understand to make sense of a pair of results. (c) Describe what you would do *during* the four hours, beyond running the two configurations, and why that work has to happen then rather than afterwards. (d) Give three of the six checklist questions you would answer before circulating any figure, and what each one would take.

2x2 — access to the run, and whether you use it

COLUMNS: IS ANY ANALYSIS PERFORMED AT ALL? →

	No analysis	Analysis performed
YOU CAN OBSERVE the system during the run	Passive benchmarking The evidence is available and nobody looks at it: pick a tool, run it with options, make a slide deck, hand it to management. <i>Move:</i> this is the cell to leave, and it is cheap to leave — the tools are installed and the load is already applied.	Active benchmarking Analysis of the system while the load runs: confirm what is really being tested, and find the true limiter. <i>Move:</i> hold the load in steady state, work down the stack until the numbers agree, and publish the limiter with the result.
YOU CANNOT — the run is over, or was never yours	A number and nothing else Bare results with no analytical detail, which can be a sign of inexperience — and this is usually where an investigation starts, not where it ends. <i>Move:</i> if it is yours, publish the assumptions you could not check. If it is someone else's, work the question lists.	Statistical analysis Analysis of the results after the run: selection, execution and collection, then interpretation. The realistic approach on expensive contended hardware. <i>Move:</i> collect far more than you think you need while on the machine; the forensics happen later. It establishes reproducibility, never correctness of target.

Read it as: the row is your access and it expires; the column is your choice and it does not. Both left-hand cells are the same failure — no analysis — and the top-left is the costly one, because the window was open.

CARRY-OUT RULE FROM THIS PART

“Why not double?” is the whole method in three words. Ask it of every benchmark result, including your own, including the ones that came out well. Answering it forces you to name the limiter; naming the limiter forces you to check whether the limiter is the thing you meant to test; and that single chain catches the majority of Part 2 before it ever reaches a slide.

Concept sketch — the whole competency, end to end

Four named groups, in the order the story runs: **THE NUMBER** → **THE LIES** → **THE KIND** → **THE HABIT**. Every node carries a question. Cover the answers, walk the map, and each answer hands you the next question.

Legend: bold = a group heading

reversed = the one group to remember

Q = retrieval cue

The four groups run in order: **THE NUMBER** — what a result is · **THE LIES** — how it goes wrong · **THE KIND** — what to run · **THE HABIT** — how to catch it.

GROUP 1 - THE NUMBER . what a benchmark result actually is

A BENCHMARK TELLS YOU ONE THING: how fast this system ran THIS BENCHMARK. everything else is your work.

Q why is "50,000 ops/sec" an incomplete claim?

THE PURPOSE sets the standard: system design . proof of concept . tuning . development . capacity planning . troubleshooting . marketing

Q which purpose asks a different question from the rest?

SIX QUALITIES repeatable . observable . portable . easily presented . realistic . runnable
+ price/performance before a purchase, priced as the five-year capital cost

Q which quality is load-bearing for group 4, and why?

THE FOUR QUESTIONS what is tested . what is the limiting factor . what perturbations . what may be concluded

Q which three lose most of their evidence when the run ends?

GROUP 2 - THE LIES . sixteen of them, sorted two ways

MEASUREMENT WRONG, BY ACCIDENT (ten of the sixteen)
casual (A/B/C) . blind faith . complex or single-threaded tools . wrong thing . wrong environment . errors . variance . perturbations . multiple factors . friendly fire

Q state the A/B/C formula and give the disk example

CLAIM WRONG, NUMBER RIGHT

no analysis . the paradox ($0.5^3 = 12.5\%$) . benchmarking the competition . lying by omission . benchmark specials

Q what did the standards bodies do about specials?

DELIBERATE FABRICATION . rare or nonexistent

Q why is suspecting fraud usually an evasion?

GROUP 3 - THE KIND . realism trades against analysability

MICRO-BENCHMARK artificial on purpose; roots-causes fast
design = 5 tests x working set x thread count
working set << RAM -> file system ; >> RAM -> disks
threads single -> clock speed ; saturating -> max

Q why is being obviously artificial an advantage?

SIMULATION built from workload characterization
stateless (independent) vs stateful (Markov model)
and it AGES as customer patterns move

Q what keeps an obsolete simulation alive?

REPLAY real operations, but a recording cannot REACT

Q what did the disk-level capture hide, and at what cost?
INDUSTRY STANDARDS independent, documented, disclosed,
auditable; your job shrinks to "which one is us?"
Q which method answers that question?

GROUP 4 - THE HABIT . analyse the system WHILE it runs

PASSIVE = pick tool, run it, make slides, hand them over

ACTIVE = observe the system under load. the ONLY way to
establish what was really measured.

Q name the layers walked in the case study, in order

PROFILE what you don't expect . SWEEP resources until a
limit is found . RAMP load and watch LATENCY .

SANITY-CHECK the arithmetic against known bandwidth

Q 990,000 at 5 ms or 1,000,000 at 50 ms - which, and why?

STATISTICAL ANALYSIS is the AFTERWARDS method. it proves
reproducibility, never correctness of target.

Q what does a perfectly reproducible cached test prove?

THE CHECKLIST why not double? . did it break limits? .

did it error? . does it reproduce? . does it matter? .

did it even happen?

Q which one is the limiter question in disguise?

SECTION 6 CONTINUED · THE TRANSFERABLE CARD

Master 2×2 — the decision card you can carry to other problems

Two questions decide whether a number is evidence or decoration, and the second is the one almost everybody skips.

	No — no limiter established	Yes — you can name and quantify the limiter
You have NOT checked what was actually MEASURED	A number, and nothing more The label on the tool is doing all the work, and no component was shown to be at its limit. <i>Move:</i> assume nothing here is true yet. Two cheap actions change the category immediately: do the arithmetic against known bandwidths, and re-run it long enough to watch. If neither is possible, publish the unchecked assumptions with the number so the gap is visible rather than implied.	A confidently wrong result You found something at 100% and named it — but you never established that the operations were what you thought. This is the most persuasive failure in the whole subject, because it comes with an explanation attached. <i>Move:</i> check the operations, not the saturation. The case study's tool was genuinely limited by something real; it was writing one byte at a time into memory. A named limiter on an unverified workload is a well-supported answer to the wrong question.
You HAVE checked what was actually MEASURED	Honest, but not finished You know what the operations are, and nothing hit a limit — which means the system was never driven to its limit and your figure is a floor, not a ceiling. <i>Move:</i> ramp the load until something saturates, watching latency as you go, and report the last point whose latency a customer would accept. “Why not double?” is the only question outstanding, and it has an answer.	A result worth quoting You can say what the workload was, and what stopped it going faster — and the two are consistent with each other. <i>Move:</i> publish both together, permanently attached. State the configuration alongside them, and say what may and may not be concluded. This is the only cell from which a comparison between two systems means anything, because it is the only one where you know the two numbers are about the same thing.

Read it as: the row is the question people skip and the column is the question they think they have answered. The card generalises well beyond benchmarking — any measurement handed to you can be interrogated with the same pair, *what exactly did you measure* and *what stopped it going higher*, and a number that cannot answer both is not yet evidence.

THE THUMB RULE, IN ONE SENTENCE

Before you believe a number, ask what it actually measured and what stopped it going higher.

If the answer to the first is “whatever the tool is called” and to the second is “nothing in particular”, you are holding a slide, not a result. It travels intact. A drug trial reporting an improvement without saying what was scored, or which arm ran short of participants; a school league table whose measure is exam passes and whose limiter is who was permitted to sit them; a fuel-economy sticker measured on a rolling road at a fixed temperature, limited by the test cycle rather than by the engine. In all three the number is real, and useless until you can name the thing measured and the thing that capped it.

Symptom → diagnosis → move

Cover the middle and right columns and work down the left.

Running one yourself

WHAT YOU OBSERVE	MOST LIKELY DIAGNOSIS	WHAT TO DO
The run is finished and now you want to know what it measured.	Passive benchmarking — the evidence expired with the load.	Re-run it, ideally in steady state, and analyse while it is live. If you cannot, publish the assumptions you were unable to check alongside the number.
Nothing reached 100% utilization during the run.	You did not drive the system to its limit.	Ramp the load higher. The resource sweep offers only two outcomes: something saturated, or you are not pushing hard enough.
The result is exactly what one thread can do.	Benchmarking the benchmark — single-threaded tool software.	Check the tool's concurrency before the target's. A tool that cannot saturate the system is measuring itself.
A “disk” test reports numbers far above any plausible device rate.	It is testing the file system cache, not the disk.	Walk down the stack: disk statistics, block layer, cache statistics, file system layer. Look for I/O actually issued and completed.
The disks are idle and there is kernel CPU time.	Something in software is limiting it — a resource control, a throttle, a default.	Profile it even though you do not expect the CPU to matter. A function name is often enough of a clue; you may not need to read the code.
Repeats of the same test disagree with each other.	Perturbations, or a run that is too short.	Lengthen the run — minutes, never under one second — repeat it, and examine the standard deviation. Collect the latency distribution, not just total runtime.
Two systems were compared and one won.	Possibly several factors changed, not one.	Enumerate every difference: network path, hops, congestion, tunables, versions, the host underneath a cloud instance. Test several instances and take the median.
Throughput rose but latency climbed sharply.	You pushed past the knee; operations are heavily queued.	Report the last point whose latency a customer would accept. A bigger number at an unacceptable latency is not a bigger number.
Your own product's result looks unimpressive.	Friendly fire — untested top configuration, an old version, limited hardware, or a limit never reached.	Ask what the bottleneck was, test the best configuration, and share results with engineering before publication.
You have four hours on contended hardware and five teams behind you.	Active analysis is still possible, but the interpretation must happen later.	Do both: analyse while live to establish what is measured, and collect every metric the overhead permits for forensic work afterwards.

Reading someone else's

WHAT YOU OBSERVE	MOST LIKELY DIAGNOSIS	WHAT TO DO
A bare number with no analysis attached.	The author assumed the result was trustworthy and final.	Treat it as the start of an investigation. Ask for the limit encountered and the analysis performed.
“We are 2× faster” with no configuration or price.	Lying by omission — technically correct, then misrepresented.	Ask what system, how many disks and of what type, which interfaces and configuration, and what it costs. Use the figure as an upper bound only.

An impossible-looking throughput over a known link.	The client cached it and the workload never reached the server.	Do the arithmetic: operations $\times$ size against the interface bandwidth, plus headers. The sum settles it without further debate.
An unusually good result from a small unknown third party.	Several were commissioned cheaply and the best kept.	Ask how many firms were engaged and what the others found. A deviation in the favourable direction is the point of the exercise.
A competitor's product performs badly in a vendor's test.	Untuned, because there was never time to tune it.	Discount it, and note that publishing it costs the vendor credibility with exactly the audience it was aimed at.
A product scores unusually well on one specific benchmark.	Possibly a benchmark special — optimised for the test.	Check whether the standard has an anti-special clause and a price rule; and check whether the winning option is one a real customer would use.
A vendor quotes a peak with no average or distribution.	The distribution is being withheld, deliberately or not.	Ask: average or peak, what is the average, what is the standard deviation or the percentiles, what is the error margin, is it reproducible.
The result is from an independent body's suite.	The good case — defined, documented, disclosed, possibly audited.	Your remaining question is fit: does this benchmark resemble our workload? Answer it by characterizing production, not by reading the marketing.
You suspect the numbers are simply invented.	Almost certainly not.	Work the list above first. Fabrication is the rarest failure and suspecting it usually substitutes for the analysis that would find the real cause.

Believing the number

WHAT YOU OBSERVE	MOST LIKELY DIAGNOSIS	WHAT TO DO
Average latency is suspiciously round and suspiciously high.	Possibly a timeout, not a response.	Check the success/fail ratio. In the case that made this famous, every request was blocked and the “latency” was the client's timeout.
The result reproduces perfectly, run after run.	Consistency, which is not correctness.	Reproducibility is one checklist question of six. A cached test that never touches the device reproduces beautifully.
The benchmark applies a steady average rate.	Variance has been erased.	Check whether the real workload bursts. Bursts queue other work and cause production issues a flat average never reproduces; consider driving state transitions instead.
Nobody can say what limited the run.	No limiter was established.	Ask “why not double?” Until that has an answer, the figure is a floor rather than a ceiling.
The benchmark tests calls your application never makes.	Irrelevance — testing the wrong thing.	Ask “does it matter?” and settle it by characterizing the production workload.
A tool's own results changed between versions.	The tool was fixed, or its buffering behaviour changed.	Accept that old and new results are no longer comparable, and say so rather than plotting them on one line.
A trend line spans several years of one internal benchmark.	The simulation has aged; it models the workload of the year it was written.	Decide openly between comparability and relevance. Keep the history, relabel it, and start a new series rather than pretending one line.
Only top speeds have ever been measured.	Sunny day testing only.	Add cloudy day runs: contention, perturbations and workload variance, deliberately induced.

Reverse mapping — you see a technique, what is it there for?

The other direction. Use this when auditing an approach rather than diagnosing a result: for each thing you find someone doing, say what it was buying and what it does *not* buy.

WHAT YOU FIND PEOPLE DOING	WHAT IT COUNTERS	WHAT IT DOES <i>NOT</i> COVER
Analysing the system while the benchmark runs	Nearly the whole accidental-measurement quadrant: wrong thing tested, errors, tool limits, unnoticed throttles, perturbations.	Claim errors. It cannot stop someone omitting the price or generalising your result into a slogan afterwards.
Profiling software you don't expect to matter	Limits imposed by software you were not looking at — resource controls, throttles, defaults that differ between systems.	Anything not consuming CPU. A resource sitting idle behind a lock will not appear.
Sweeping resources for utilization, saturation and errors	Publishing a number that no component's limit explains.	Whether the operations counted were the ones you meant. It finds <i>a</i> limit, not the right workload.
Characterizing the production workload	Testing the wrong thing, and choosing an irrelevant industry benchmark.	Anything about the benchmark run itself; it never touches the system under test.
Ramping load in small increments	Reporting a floor as a ceiling, and guessing at the scalability profile.	Whether the number is honest at that load — which is why latency must be measured alongside.
The arithmetic sanity check	Results that would require a component to exceed a known limit — usually client caching.	Results that are merely wrong rather than impossible. Passing the check is not evidence of correctness.
Longer runs and repeated runs	Perturbations, interrupt and scheduling effects, cache warmth, and run-to-run noise.	Any systematic error. A biased measurement repeated fifty times has a beautiful standard deviation.
Collecting the full latency distribution	Outliers hidden inside a total runtime, and averages that describe nothing that happened.	The cause of the outliers, which still needs tracing.
Writing your own short benchmark	Complexity that hinders analysis, and tools that do more than you need.	Your own bugs, and compiler optimisations that delete work you meant to measure.
Publishing the unchecked assumptions	Numbers presented as final when no analysis was done.	The analysis itself. It converts an implicit gap into an explicit to-do list, which is honest but not finished.
Using an independent body's suite	Incomparable vendor claims, undisclosed configurations, and having to run everything yourself.	Fit to your workload, and — depending on the body — price.
Statistical analysis of results afterwards	Making sense of a pair of results: variation, error margins, distributions, reproducibility.	What was measured. That evidence exists only while the load is applied.

Numbers worth remembering

NUMBER	WHAT IT IS
6 qualities, +1	A good benchmark is repeatable, observable, portable, easily presented, realistic and runnable. Before a purchase, add the price/performance ratio — price quantified as the five-year capital cost of the equipment.
4 questions	Benchmark analysis: what is being tested; what the limiting factors are; what perturbations might affect it; what conclusions may be drawn. Three of the four need the system under load.
16 failures	The catalogue in Part 2. Ten are accidental measurement errors; the last, cheating, is described as rare or nonexistent.
$0.5^3 = 12.5\%$	The benchmark paradox: a coin-flip product asked to win three benchmarks wins all three one time in eight. Hence “if your chances are 50/50, you'll usually lose”.
≥ 1 second	The floor for a benchmark run's duration. The working target is minutes, not seconds , to iron out perturbations; then check the standard deviation across repeats.
2%	The transaction-processing body's price requirement: the total price must be within 2% of what a customer would actually pay. Introduced against pricing-based benchmark specials.
1993	When the term <i>benchmark special</i> came into use, over a database option added to inflate results on a transaction benchmark. The answer was an anti-benchmark-special clause in the specification.
500 / 50 per second	The illustrative disk workload averaging 500 reads and 50 writes per second — simulated at those rates or at the 10:1 ratio, and thereby erasing write bursts that arrive roughly every 10 seconds .
40 / 7 / 19 / 1 %	An example characterized file-server workload: reads 40%, writes 7%, getattr 19%, readdir 1% — the kind of measurement a simulation is built from.
1972 and 1984	Two well-known CPU benchmarks are simulations of their own eras — scientific workloads and integer workloads of the time. Simulations age.
10K → 7,200 rpm, 16×	The replay case: slower disks but sixteen times the file system cache. The disk-level replay showed a regression; the real workload would have improved from cache. The sale was lost.
1985	The paper that argued for measuring the price/performance ratio, detailed three vendor-executable benchmarks (Sort, Scan, DebitCredit), and proposed transactions per second as a cross-vendor measure — the way miles per gallon is for cars.
5 sample tests	The file system micro-benchmark design: sequential 512-byte reads (maximum realistic IOPS), sequential 1 MB reads (read throughput), sequential 1 MB writes (write throughput), random 512-byte reads (effect of random I/O), random 512-byte writes (effect of rewrites) — each multiplied by working set size and thread count.
4 MB in 28 s	The 1981 network-stack analysis: 4 megabytes between user processes on two machines, 1024-byte records in 1068-byte packets, 28 seconds including setup and teardown, giving 1.2 Megabaud user-to-user.
20 / 10 / 30 / 25 / 15 %	Where that system time went: Ethernet handling, IP processing, TCP processing, checksumming, and user system-call handling — with no single part dominating . The sender was CPU saturated; the receiver had about 30% idle .
739 → 739,000	The case study's first test scored 739 Kbytes/sec per character. If that were truly I/O per character it would mean 739,000 I/O per second — the sanity check that started the investigation.
1,176,936 / 668,839	What it was really doing: over a million write calls at the file system layer, of which 668,839 were 1 byte each, buffered in cache. Reads: 13. Disk I/O issued: none.
62.17%	The share of CPU samples in a file system I/O throttle during a disappointing disk benchmark — a resource control left at its default on the new system but not the old.
~500,000 reads/s	The ramping-load example's peak: 8 Kbyte random reads on a cached file, threads added one by one, over 3.5 Gbytes/s at peak, ultimately limited by server CPU.

990,000 vs 1,000,000	990,000 IOPS at 5 ms average, or over a million at more than 50 ms with everything heavily queued. The number you publish is 990,000 .
50,000 × 8 KB	= 400,000 Kbytes/s, plus headers — over 3.2 Gbit/s on a single 1 Gbit/s interface. Impossible, so the benchmark tested client caching.
119 Mbytes/s	Roughly what a 1 Gbit/s interface reaches in practice. Hence 120 Mbyte/s may be fine, 200 is possible only if traffic in both directions was summed, and 350, 800 and 1,150 are clearly bogus.
6 checklist questions	Why not double? Did it break limits? Did it error? Does it reproduce? Does it matter? Did it even happen?
4 steps / 3 phases	Passive benchmarking has four steps (pick, run, slides, hand over). Statistical analysis has three phases (selection, execution and collection, interpretation).

Glossary

TERM	MEANING
Benchmarking	Testing performance in a controlled manner, so choices can be compared and limits understood before they are met in production. A benchmark tells you only how fast the system ran the benchmark.
Benchmarking	Benchmarking to determine maximum product performance for promotional use. A legitimate purpose that answers a different question from capacity planning.
Price/performance ratio	Performance per unit of cost; the extra quality required when comparing systems with intent to purchase. Price may be quantified as the five-year capital cost of the equipment.
Casual benchmarking	Fire-and-forget benchmarking: you benchmark A, actually measure B, and conclude you have measured C.
Argumentum ad populum	“Appeal to the people” — the misconception that popularity equals validity. The named fallacy behind blind faith in a well-known benchmark.
Benchmarking the benchmark	When the reported result is limited by the benchmark software itself. A common cause is single-threaded benchmark software.
Perturbation	External activity affecting a result: a scheduled backup, a monitoring agent, an unseen tenant on the same host, or short-run effects such as interrupts, scheduling decisions and cache warmth.
Markov model	A model representing requests as states and measuring the probability of transitions between them; the instrument for simulating variance and stateful workloads rather than a flat average mix.
Benchmark paradox	If your product's chances of winning a benchmark are 50/50, you will usually lose — because customers run several and want you to win them all.
Friendly fire	Underselling your own product by benchmarking a suboptimal configuration, an old version, limited equipment, or never driving the system to its true limit.
Lying by omission	A benchmark result that is technically correct but misrepresented by leaving out the price, the configuration, or the scope of the claim. Not the same as fake numbers.
Benchmark special	Engineering the product to score well on a popular or industry benchmark while disregarding actual customer performance. Also called optimising for the benchmark.
Micro-benchmark	An artificial workload testing one particular type of operation. Simple, repeatable, quick to root-cause, and deliberately not mistakable for a real workload.
Simulation (macro-benchmark)	A benchmark imitating a customer application workload, usually built from workload characterization of production.
Stateless / stateful simulation	Stateless: each request is independent, its type drawn randomly from measured probabilities. Stateful: each request depends on client state, at minimum the previous request.
Replay	Replaying a captured trace log against the target. Problematic because a recording cannot respond naturally to a server whose characteristics and delivered latency have changed.

Working set size	The size of the data being accessed. Much smaller than main memory tests the file system software; much larger minimises cache and drives the test toward disk I/O.
Sunny day testing	Benchmarks focused on top speeds. Its complement is cloudy or rainy day testing: non-ideal situations including contention, perturbations and workload variance.
Industry-standard benchmark	A benchmark from an independent organisation: well defined, documented, executed under guidelines, and usually requiring full disclosure of the configured environment, which may be audited.
TPS / MIPS / FLOPS	Cross-vendor measures — transactions, millions of instructions, and floating-point operations per second. Instruction-based measures are hard to compare across processor architectures; industry suites usually report a custom metric that compares only with itself.
Passive benchmarking	The anti-methodology: run the benchmark, ignore it until it finishes, collect the data. No analysis is performed.
Active benchmarking	Analysing performance while the benchmark runs, using observability tools — to confirm what is really being tested and to find the true limiter of the system or of the benchmark.
Statistical analysis	Analysis of the results after the run: selection, execution and collection, interpretation. Establishes reproducibility, variation, error margins and distributions — never what was measured.
Ramping load	Adding load in small increments and measuring delivered throughput until a limit is reached, producing a scalability profile.
Sanity check	Checking whether a result would have required a component to exceed a known limit — network, controller, interconnect bandwidth or disk IOPS.
Steady state	A benchmark configured to hold a stable load indefinitely, so analysis can continue over hours or days.
Limiting factor	What stopped the result going higher. A number quoted without one is a floor, not a ceiling — and answering “why not double?” is the same question.

Answer key

PART 1 · Q1 — THE TEN-MINUTE COMPARISON AND THE THURSDAY REVIEW

(a) The four are: what is being tested; what the limiting factor or factors are; any perturbations that might affect the results; and what conclusions may be drawn. This exercise has established none of them. The tool's name is standing in for the first, "looked busy" is not a limiting factor — the resource sweep offers only two outcomes, something at 100% or a system not driven to its limit, and neither has been shown — nothing was checked for perturbations, and the conclusion (1.8×) has been drawn without any of the three that would license it. "No errors reported" is worth something but is not one of the four. (b) Because three of the four questions can only be answered from the system while the load is applied, and by Thursday the machines are idle. This is the staffing mistake in its usual form: junior staff execute the benchmarks and performance experts are brought in to explain the results afterwards. The correction is scheduling rather than skill — engage the expert *during* the run, so they can analyse the live system with drill-down to quantify the limiter. If Thursday is genuinely all there is, the useful output is a list of the assumptions nobody had time to check, published with the result as a to-do list for verification. (c) The purpose here is a purchase, so the required extra quantity is the **price/performance ratio**, where price can be quantified as the five-year capital cost of the equipment. Further questions worth putting: what was the limiting factor in each run; what was the configuration of each system, and does either resemble what we would actually buy and run; was the workload representative of ours, and how was that established; how long did each run last and how many times was it repeated; what were the success and failure counts; is this an average or a peak, and what is the distribution.

PART 1 · Q2 — THE BARE FIGURE AND THE DECOMPOSED ONE

(a) The second result answers the four questions and the first answers none. *What is being tested*: the second states the payload size, the batch size and the volume moved, precisely enough to reproduce; the first says only "operations", which could be anything of any size. *What the limiting factor is*: the second names it — the producer's network interface at 96% of line rate throughout, with the consumer's processors never above 40%, so one component was at its limit and the other plainly was not; the first offers nothing. *Perturbations*: the second's decomposition of producer processor time across four named areas, none dominant, shows nothing unexpected was consuming the machine during the run; the first gives no basis on which to check. *What may be concluded*: the second stops at what the measurement supports and draws no wider claim; the first's single number is already a claim about a whole cluster. (b) Because a network interface at 96% of line rate makes the sustained rate a statement about **that link**, not about the message bus. The same software on the same hosts behind a faster interface would give a different figure, so the number cannot be quoted as a property of the bus, of the consumer, or of the pair — and for the bus it is a floor rather than a ceiling. Naming the limiter is what converts a measurement into a claim with a subject. (c) The strongest version of their case: a headline figure exists to be compared with other headline figures, decomposition costs analyst time that a purchasing conversation will not read, and if the cluster genuinely served 1.4 million operations then it did so regardless of where the processor time went. What you would still require before quoting it: what the operations actually were and at what size; what limited the run, established by observing the system rather than asserted; whether any component reached 100%, since if none did the figure is a floor rather than a ceiling; the error and timeout counts; the duration and the number of repeats with their variation; and the configuration, so the reader knows which machine the number is about. Absent those, the honest form is the number plus an explicit list of the assumptions nobody checked.

PART 2 · Q1 — THE TWO BROKERS

(a) Four named failures. **Complex benchmark tools**, in its specific form *benchmarking the benchmark*: a single-process client means the reported result may be the tool's own ceiling rather than either broker's — measurement wrong. **Changing multiple factors**: one broker acknowledged writes durably and the other did not, so the two runs differ in more than the product — measurement wrong. **Ignoring errors**: roughly 8% of B's requests errored and were counted as completed operations, and errors perform differently from normal operations and skew the result — measurement wrong. **Ignoring perturbations**: an index rebuild ran on the B host through the last six seconds of a twenty-second run — measurement wrong. All four are in the accidental-measurement quadrant, which is where ten of the sixteen live. (b) Two near-free remedies. For perturbations: make the runs longer — minutes instead of seconds — with the rule that a benchmark's duration should not be shorter than one second, then run it several times and examine the standard deviation, which should be as small as possible. For errors: check the operation success/fail ratio before reading any throughput figure; the checklist puts this as "did it error?" and "did it even happen?". (c) The slide commits **numbers without analysis**: bare results with no analytical detail, which can be a sign that the author is inexperienced and has assumed they are trustworthy and final. Every benchmark number should be accompanied by a description of the limit encountered and the analysis performed. Failing that, the slide should have carried the list of assumptions nobody had time to check — that the tool is not the bottleneck, that the two configurations are equivalent, that the operations succeeded, that nothing else was running — which then becomes the verification to-do list if the decision turns out to matter.

PART 2 · Q2 — FOUR BENCHMARKS AND THREE PROPOSALS

(a) $0.5 \times 0.5 \times 0.5 \times 0.5 = 0.0625$, so **6.25%** — about one chance in sixteen. The rule is the benchmark paradox: “if your product's chances of winning a benchmark are 50/50, you'll usually lose”, because customers who want to be sure run several and expect you to win them all, and the more benchmarks the worse the chances. (b) Proposal (i) is **benchmarking the competition**, and in its cheap-third-party form it is also the **misleading benchmarks** variant where several firms are engaged not for expertise but for a fast inexpensive result and the best is kept. It backfires because customers use a product for months and tune it, while you have days — so you can only produce untuned and therefore unrealistic results, and the competitor's customers, who are the target of the exercise, will recognise them, costing your company credibility with exactly the people it was trying to impress. Proposal (ii) is a **benchmark special**: engineering the product to score on the benchmark while disregarding actual customer performance. The **transaction-processing body** — the one whose focus is price/performance — answered it with specification rather than argument, adding an anti-benchmark-special clause prohibiting all implementations that improve benchmark results but not real-world performance or pricing, and accompanying it with a price requirement that the total price be within 2% of what a customer would actually pay, which closes the parallel trick of quoting deep discounts nobody would receive. (c) Proposal (iii) commits the pitfall that **ignoring benchmarks is itself a benchmarking failure**. The intervention that genuinely improves the odds is to stop being a coin flip and to make sure your own results are the best your product can honestly produce — the friendly-fire remedy: test the top-performing system and configuration, drive it to its true limit and be able to say what the bottleneck was, benchmark the current version on the best available equipment, and share the results with engineering before publication so they can spot what was misconfigured or left off.

PART 3 · Q1 — REPLAYING BLOCK-LEVEL TRAFFIC AT A NEW STORAGE TIER

(a) It will show a regression, and the regression will be an artefact. The trace was captured at the **block device** layer, which means it records the requests the *old* system's cache failed to absorb. Replaying those against the new tier asks its slower spinning media to service, from disk, precisely the traffic the new tier's four-times-larger cache was bought to absorb — so the new tier is measured on its weakest component with its strongest one bypassed. The real production workload would have improved, returning largely from cache, which replaying device-level events cannot simulate. It is the storage-upgrade case exactly: 10K rpm disks replaced by 7,200 rpm disks with sixteen times the file system cache, a replay that showed a regression, and a lost sale; it is a case of **testing the wrong thing**. (b) Even at the right level, a replay disappoints because a **recording cannot react**: when the server's characteristics and delivered latency change, the captured client workload is unlikely to respond naturally to those differences, which may prove no better than a simulated workload — and when too much faith is placed in it, things can get worse. Subtle timing effects can also disturb it. (c) Every micro-benchmark test is multiplied by **working set size** and **thread count**. Working set much smaller than main memory caches entirely, so the file system software is what you investigate; much larger than main memory minimises cache and drives the test toward disk I/O. Thread count, assuming a small working set: single-threaded tests file system performance at the current CPU clock speed; enough threads to saturate all CPUs tests the maximum of system, file system and CPUs together. To answer “does the extra memory help?” you need both settings of the working set on both tiers, and in particular a working set sized to fit the new tier's memory but not the old tier's — that is the cell where the extra cache is the only thing that differs. Run single-threaded to attribute the result to the software path and saturating to establish the ceiling.

PART 3 · Q2 — THE SIX-YEAR-OLD INTERNAL BENCHMARK

(a) It is a **simulation**, also called a macro-benchmark: a benchmark imitating a customer application workload, normally built from workload characterization of the production environment. The weakness this is an instance of is that **customer usage patterns change over time**, so simulations require updating and adjusting to stay relevant — and there is resistance to that when published results based on the older version would no longer be usable for comparison with the new one. (b) What is being traded is **comparability against relevance**: two years of quarterly results form a series that only holds together while the benchmark is unchanged, and the benchmark is only worth running while it resembles the traffic. Either way, keep the two years and relabel them as measurements of the old workload — they remain valid for comparison with each other and invalid as statements about current performance. Do not plot old and new on one line; start a new series and say plainly which property you chose. (c) An independent body's suite offers: tests that are well defined and documented and must be executed under guidelines so results are as intended; **full disclosure of the configured environment**, which may be audited, and depending on the body either price/performance or a component and tuning list; and results that **already exist for a variety of vendors and products**, so you can compare without running anything. The question you must answer first is whether it resembles your production workload — for a current workload that is decided by **workload characterization**, measuring the real makeup of your traffic and comparing it against what the suite exercises.

PART 4 · Q1 — 400,000 READS PER SECOND OVER A 10 GBIT/S LINK

(a) The sanity check: $400,000 \text{ operations per second} \times 4 \text{ Kbytes} = 1,600,000 \text{ Kbytes/s}$, which is about 1.6 Gbytes/s, or roughly **12.8 Gbit/s** — before protocol headers — over a single 10 Gbit/s interface. That exceeds the known limit of the link, so the result is not merely doubtful, it is impossible, and the conclusion is that the figure is bogus. No further debate is needed; the arithmetic settles it. (b) The likely explanation is that the benchmark tested **client caching** and never drove the entire workload to the server, which is what results of this shape usually mean. During the run this would have been caught by observing the *server* — counting the operations actually arriving there

and comparing them with what the clients reported — which is active benchmarking rather than reading the client's summary. (c) “No component reached saturation” is a finding in its own right, because the resource sweep offers exactly two outcomes: either some hardware or software component has reached 100% utilization, or you are not driving the system to its limit. Since nothing did, the second holds, so the figure is a **floor rather than a ceiling** and no limiting factor has been established — meaning the result cannot honestly be quoted at all. The checklist puts the same point in three words: **“Why not double?”**

PART 4 · Q2 — FOUR HOURS ON CONTENTED HARDWARE

(a) **Statistical analysis**, which is the approach used where access to a large-scale system is both time-limited and expensive and many teams — sales, marketing, support, engineering, quality — want the same machine. Its three phases: selection of the benchmark tool, its configuration and the system performance metrics to capture; execution of the benchmark, collecting a large dataset of results and metrics; and interpretation of the data with statistical analysis, producing a report. It can establish consistency, variation, error margins and full distributions. It cannot establish what was actually measured — that evidence exists only while the load is applied, which is active benchmarking's job. (b) Because when benchmarking for code changes or non-regression testing it is crucial to understand the **variation** and the **error margins** in order to make sense of a pair of results. A claimed 15% difference means nothing if run-to-run variation is of the same order, and a single before-and-after pair contains no information about that variation at all. (c) During the four hours, beyond running the two configurations: analyse the system while the load is applied — confirm the benchmark is testing what it claims, sweep resources for utilization, saturation and errors until either something is at 100% or you accept the system was not driven to its limit, and profile both target and benchmark software to see whether anything unexpected is limiting it. Also collect as much performance data as the collection overhead allows, including variation, full distributions and error margins, so forensic analysis is possible afterwards; a script that dumps every available counter before, during and after each run is the standard form. It has to happen then because the machine is not yours next month, because the metrics are expensive enough that a problem found later means redoing them, and because what was measured is unrecoverable once the load stops. (d) Three of the six, with what each requires: **Why not double?** — needs the limiter, established by the resource sweep and profiling during the run. **Does it reproduce?** — needs repeated runs and their standard deviation, which is also what makes the pair of results interpretable. **Did it error?** — needs the operation success/fail ratio, since errors perform differently from normal operations and a high error rate skews the result. (Equally acceptable: **did it break limits?**, needing the arithmetic against known component limits; **does it matter?**, needing workload characterization of production; **did it even happen?**, needing confirmation at the target that the work arrived.)

Spaced-review tracker

Review at **day 1, 3, 7, 16, 35**. Write the date in the box when you pass. On a fail, reset that row to a 1-day interval and start again.

ITEM	DAY 1	DAY 3	DAY 7	DAY 16	DAY 35	FAILS
1 • What a benchmark result is a fact about						
2 • The reasons for benchmarking, including marketing						
3 • The six qualities, and the seventh before a purchase						
4 • The four questions of benchmark analysis						
5 • The staffing mistake, and why it is structural						
6 • What good analysis reads like — the 1981 example						
7 • Casual benchmarking, in its A/B/C form						
8 • Blind faith, and why auditing the code isn't enough						
9 • Numbers without analysis, and the assumption list						
10 • Benchmarking the benchmark, and its usual cause						
11 • Ignoring errors — the firewall case						
12 • Variance vs perturbation, and the remedies for each						
13 • Changing multiple factors, and the median rule						
14 • The benchmark paradox, and its arithmetic						
15 • Benchmarking the competition, and friendly fire						
16 • Lying by omission vs cheating						
17 • Benchmark specials, and the two rules against them						
18 • Micro-benchmarks, and why artificiality helps						
19 • The five sample tests, and the two multipliers						
20 • Sunny day vs cloudy day testing						
21 • Simulation: stateless vs stateful, and ageing						
22 • Replay, and why the storage case inverted						
23 • Industry standards, and what disclosure gives you						
24 • TPS, MIPS, FLOPS, and custom metrics						
25 • Passive benchmarking's four steps						
26 • Active benchmarking, and the case study's layers						
27 • Profiling what you don't expect — the throttle						
28 • The resource sweep's two outcomes						
29 • Ramping load, and the 990,000 judgement						
30 • The sanity check, and the 1 Gbit/s list						
31 • Statistical analysis, and what it cannot establish						

32 • The six-question checklist						
33 • Master matrix — what was measured × what limited it						

Final quiz — no notes, twenty minutes, write the answers

1. State what a benchmark result is a fact about and what it is not, give the six qualities of a good benchmark and the additional quantity needed before a purchase, and say how that quantity's price component is defined.
.....
2. Give the four things that must be understood about any benchmark result, say how they are best served, and explain what specifically is lost by bringing in a performance expert after the run has completed.
.....
3. State the casual benchmarking formula in its A/B/C form, give the standard example with the reason the discrepancy is so large, and say why a correctly functioning tool does not save you from it.
.....
4. Explain what should accompany every benchmark number, what to publish instead when there is no time for analysis, and give the summarised rule about how long a result deserves before you trust it.
.....
5. Distinguish ignoring variance from ignoring perturbations: define each, give an example, and give the specific remedy for each — including the minimum run duration and the statistic to check across repeats.
.....
6. State the benchmark paradox and its arithmetic, then explain why benchmarking a competitor is self-defeating and what would have to be done to do it honestly.
.....
7. Define a benchmark special, say when the term came into use and over what, and give the two specification changes the industry body introduced in response.
.....
8. Describe the design of a micro-benchmark set: the five sample file system tests with the intent of each, and the two factors every test is multiplied by, with what each setting of each factor tests.
.....
9. Compare simulation and replay: what each is, what each buys you, and what went wrong in the storage-upgrade case — naming the level of capture and the component whose improvement it hid.
.....
10. Give the four steps of passive benchmarking and define active benchmarking, then describe the case study that showed a “hard drive” benchmark testing no disk at all — the sanity check that started it, the layers checked, and the conclusion.
.....
.....